

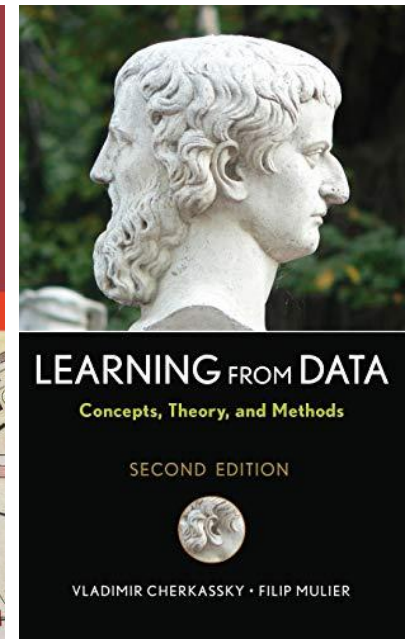
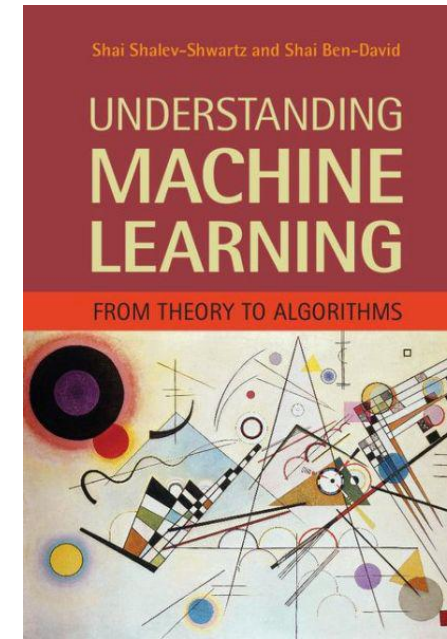
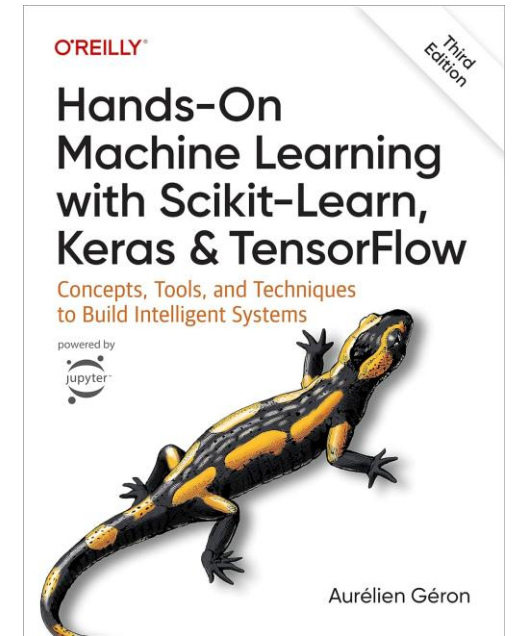
Aprendizaje Automático

Curso 2025-2026

Tema 2. Aprendizaje Supervisado



Bibliografía



- 
- Tema 1: Conceptos básicos del Aprendizaje Automático
 - **Tema 2: Aprendizaje Supervisado. Regresión y Clasificación. Modelos. Evaluación**
 - Tema 3: Ingeniería de Características y Preprocesamiento de Datos. Calidad de Datos
 - Tema 4: Aprendizaje no Supervisado
 - Tema 5: Aprendizaje Profundo. Fundamentos
 - Tema 6: Aprendizaje Profundo en diferentes tipos de datos y problemas
 - Tema 7. Aprendizaje por refuerzo
 - Tema 8: Optimización en Aprendizaje Automático. Hiperparámetros, selección de modelos y regularización
 - Tema 9: Aspectos Avanzados, retos y desafíos del aprendizaje automático



Aprendizaje supervisado

Introducción

Evaluación de modelos

Aprendizaje basado en instancias

Regresión Lineal

Modelo Naïve Bayes

Modelos basados en árboles de decisión

Modelos multclasificadores y de
descomposición



Aprendizaje supervisado

Introducción

Evaluación de modelos

Aprendizaje basado en instancias

Modelo Naïve Bayes

Regresión Lineal

Modelos basados en árboles de decisión

Modelos multclasificadores y de
descomposición

Aprendizaje supervisado

- En el **aprendizaje supervisado**, el conjunto de entrenamiento con el que se alimenta al algoritmo incluye las soluciones deseadas, denominadas etiquetas.

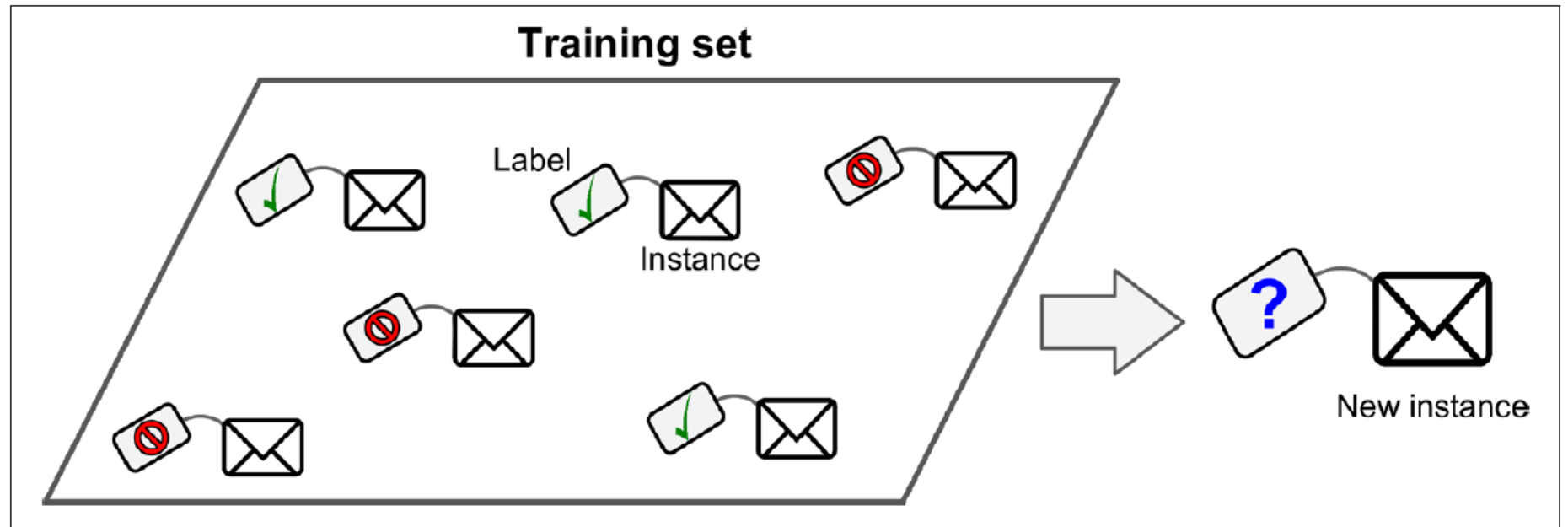
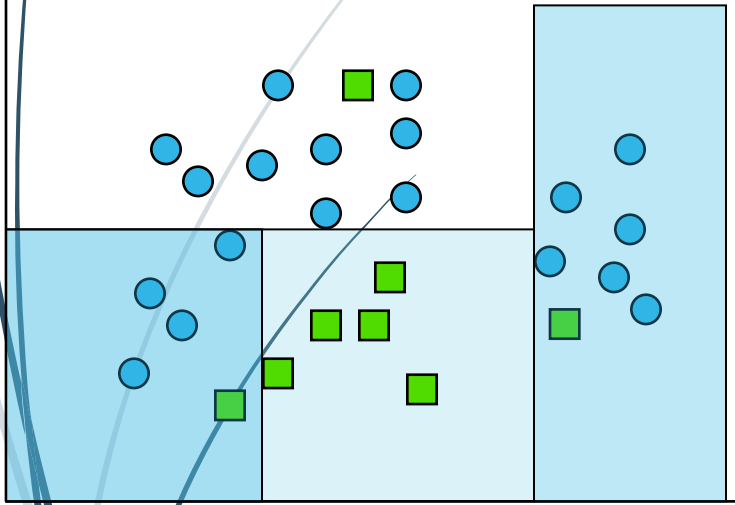


Figure 1-5. A labeled training set for spam classification (an example of supervised learning)

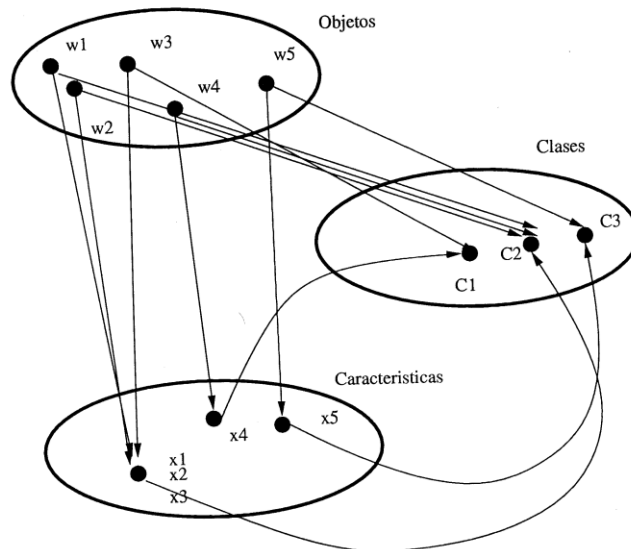
Aprendizaje supervisado

Clasificación (2 clases)



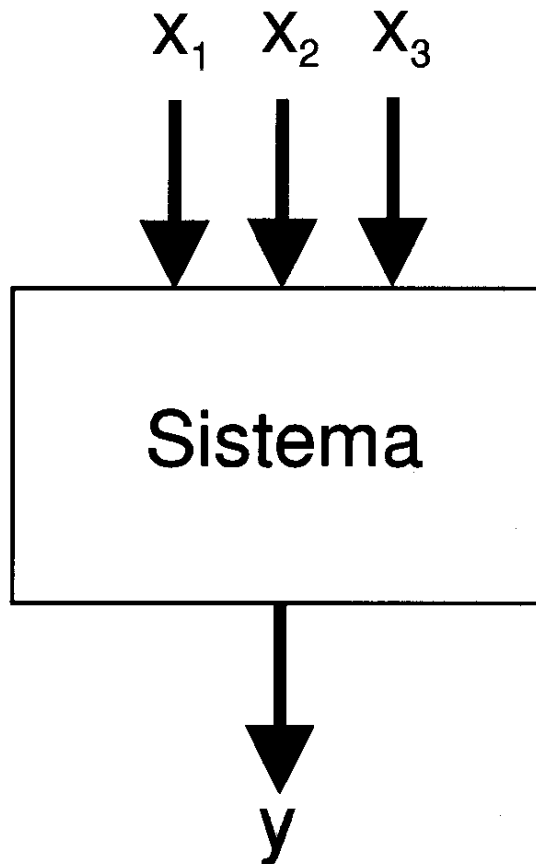
$$L(y, f(\mathbf{x}, \omega)) = \begin{cases} 0, & \text{if } y = f(\mathbf{x}, \omega), \\ 1, & \text{if } y \neq f(\mathbf{x}, \omega). \end{cases}$$

$$R(\omega) = \int L(y, f(\mathbf{x}, \omega)) p(\mathbf{x}, y) d\mathbf{x} dy$$



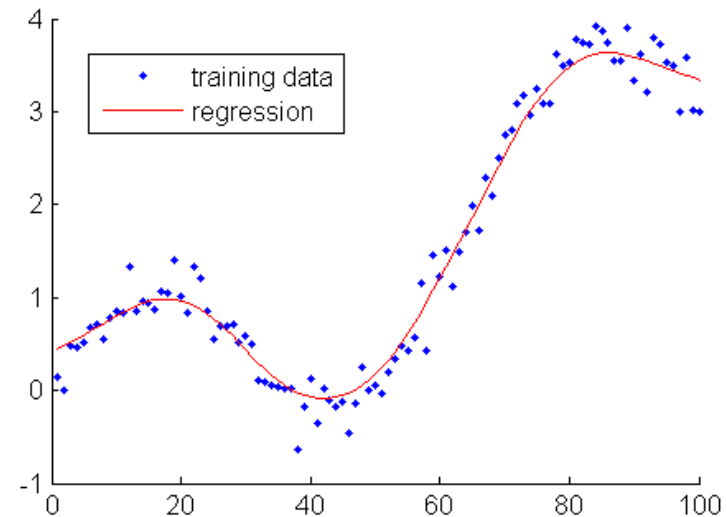
Aprendizaje supervisado

Regresión



$$L(y, f(\mathbf{x}, \omega)) = (y - f(\mathbf{x}, \omega))^2.$$

$$R(\omega) = \int (y - f(\mathbf{x}, \omega))^2 p(\mathbf{x}, y) d\mathbf{x} dy$$



Aprendizaje supervisado

- Ejemplo: Diseño de un Clasificador para *Iris*
 - Problema simple muy conocido: *clasificación de lirios*.
 - Tres clases de lirios: *setosa*, *versicolor* y *virginica*.
 - Cuatro atributos: *longitud y anchura de pétalo y sépalo*, respectivamente.
 - 150 ejemplos, 50 de cada clase.
 - Disponible en <http://www.ics.uci.edu/~mlearn/MLRepository.html>



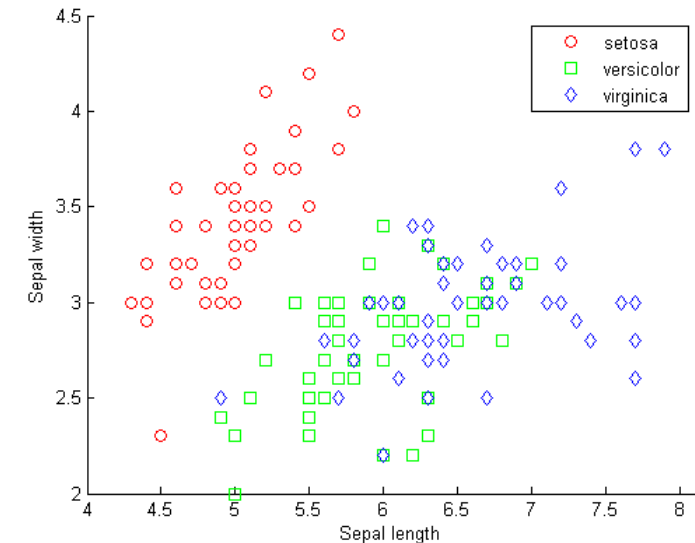
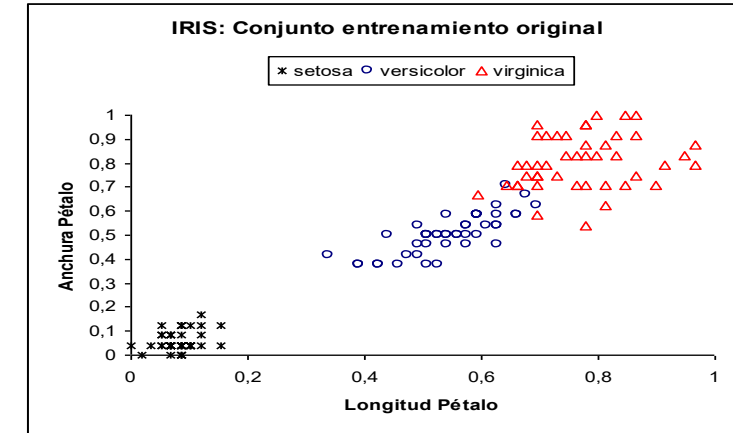
setosa



versicolor



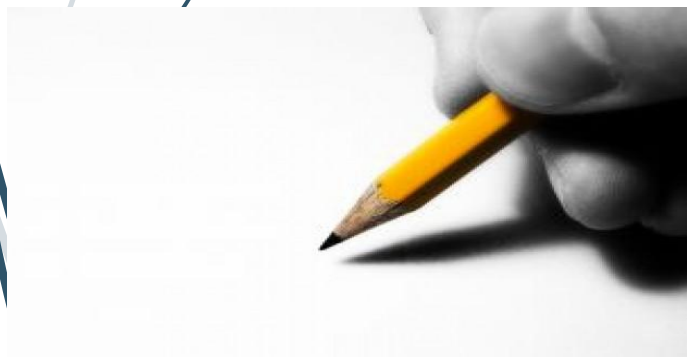
virginica



Aprendizaje supervisado

MNIST data

Handwriting recognition.
Assign a digit from 0 to 9.



Aprendizaje basado en instancias frente a aprendizaje basado en modelos

- **Aprendizaje basado en instancias:** Posiblemente la forma más trivial de aprender sea simplemente aprender a partir de ejemplos muy parecidos al nuevo. El sistema aprende los ejemplos de memoria y, a continuación, generaliza a nuevos casos utilizando una medida de similitud para compararlos con los ejemplos aprendidos (o con un subconjunto de ellos).

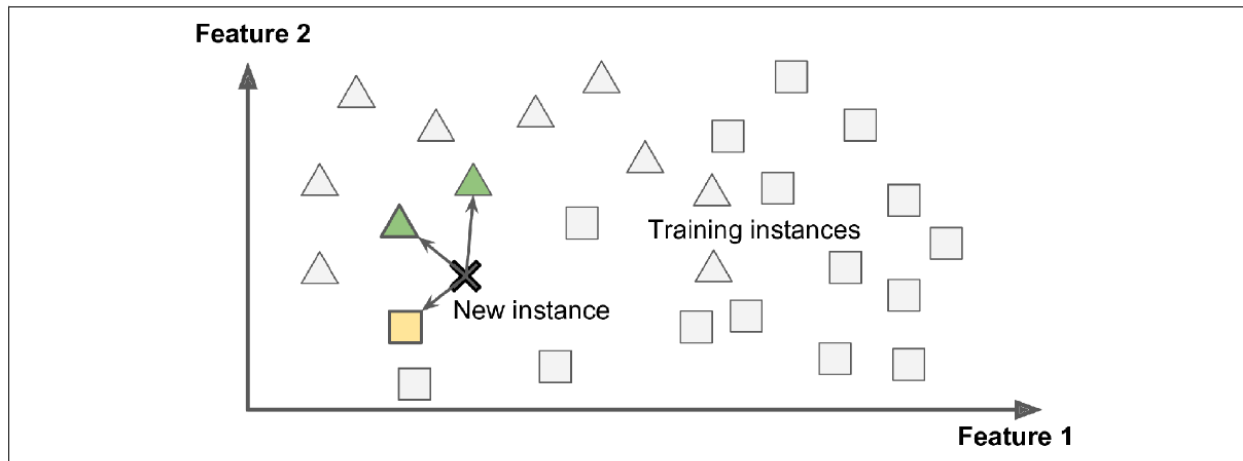
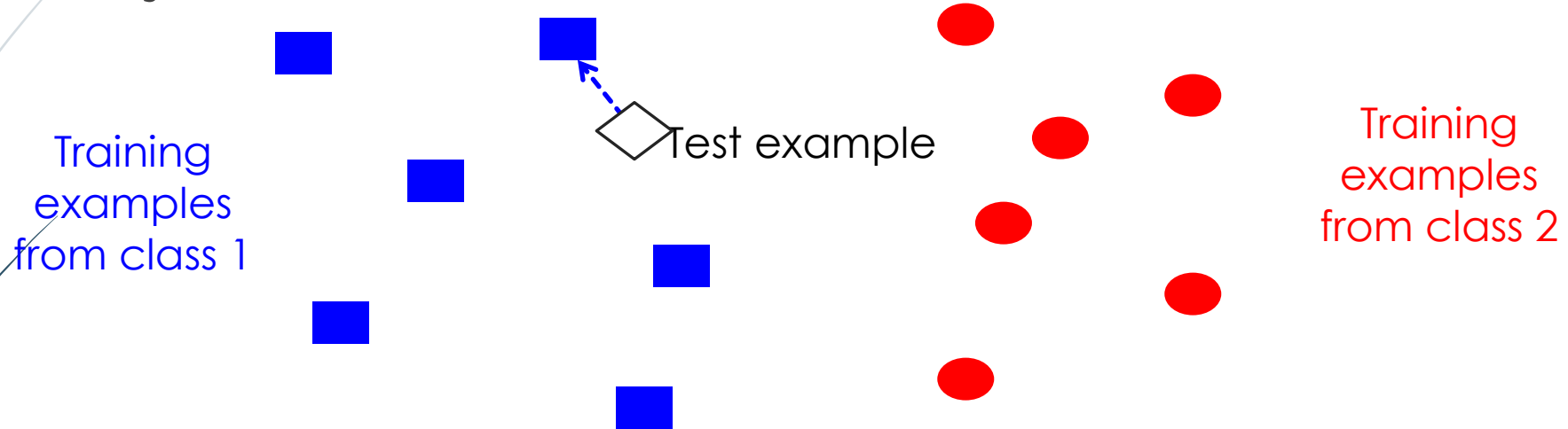


Figure 1-15. Instance-based learning

Aprendizaje basado en instancias frente a aprendizaje basado en modelos

► Aprendizaje basado en instancias



$f(\mathbf{x})$ = etiqueta del ejemplo de entrenamiento más cercano a $\mathbf{x}$

- Todo lo que necesitamos es una función de distancia para nuestras entradas.
- ¡No se requiere entrenamiento!

Aprendizaje basado en instancias frente a aprendizaje basado en modelos

➤ Otra forma de generalizar a partir de un conjunto de ejemplos es construir un modelo de esos ejemplos y utilizarlo para hacer predicciones. Es lo que se denomina **aprendizaje basado en modelos**.

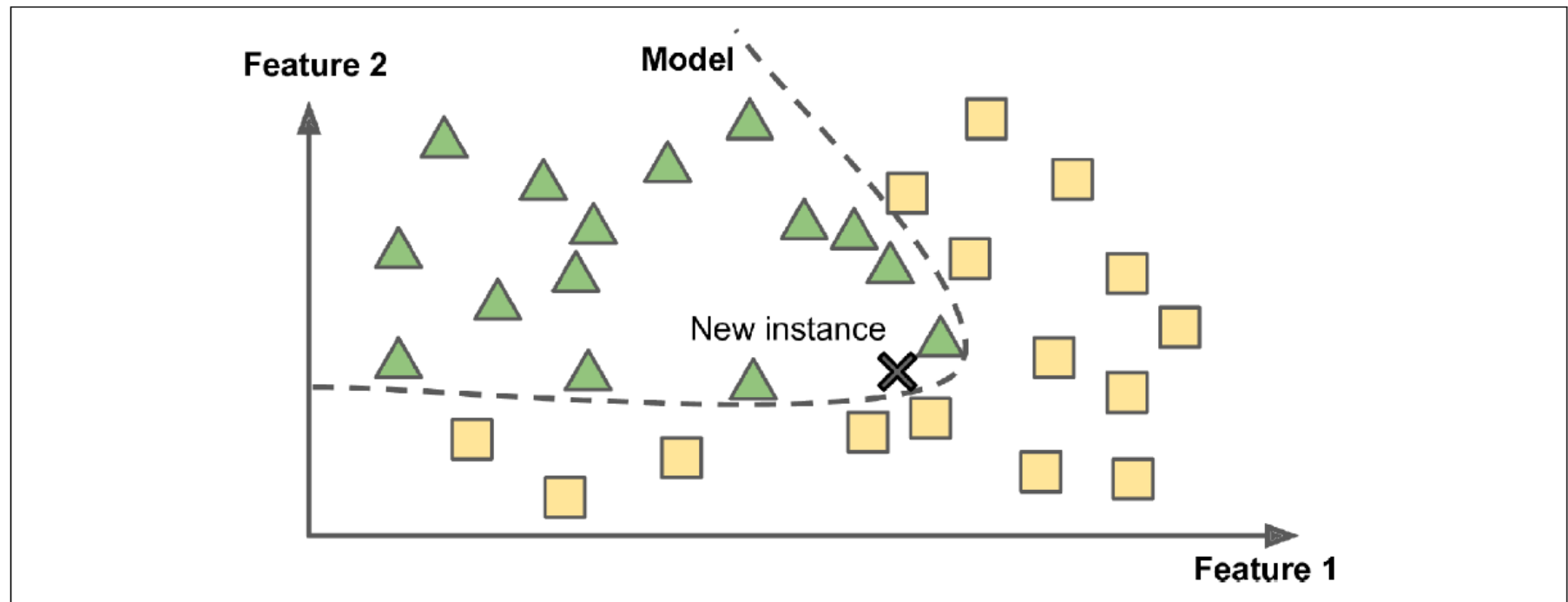
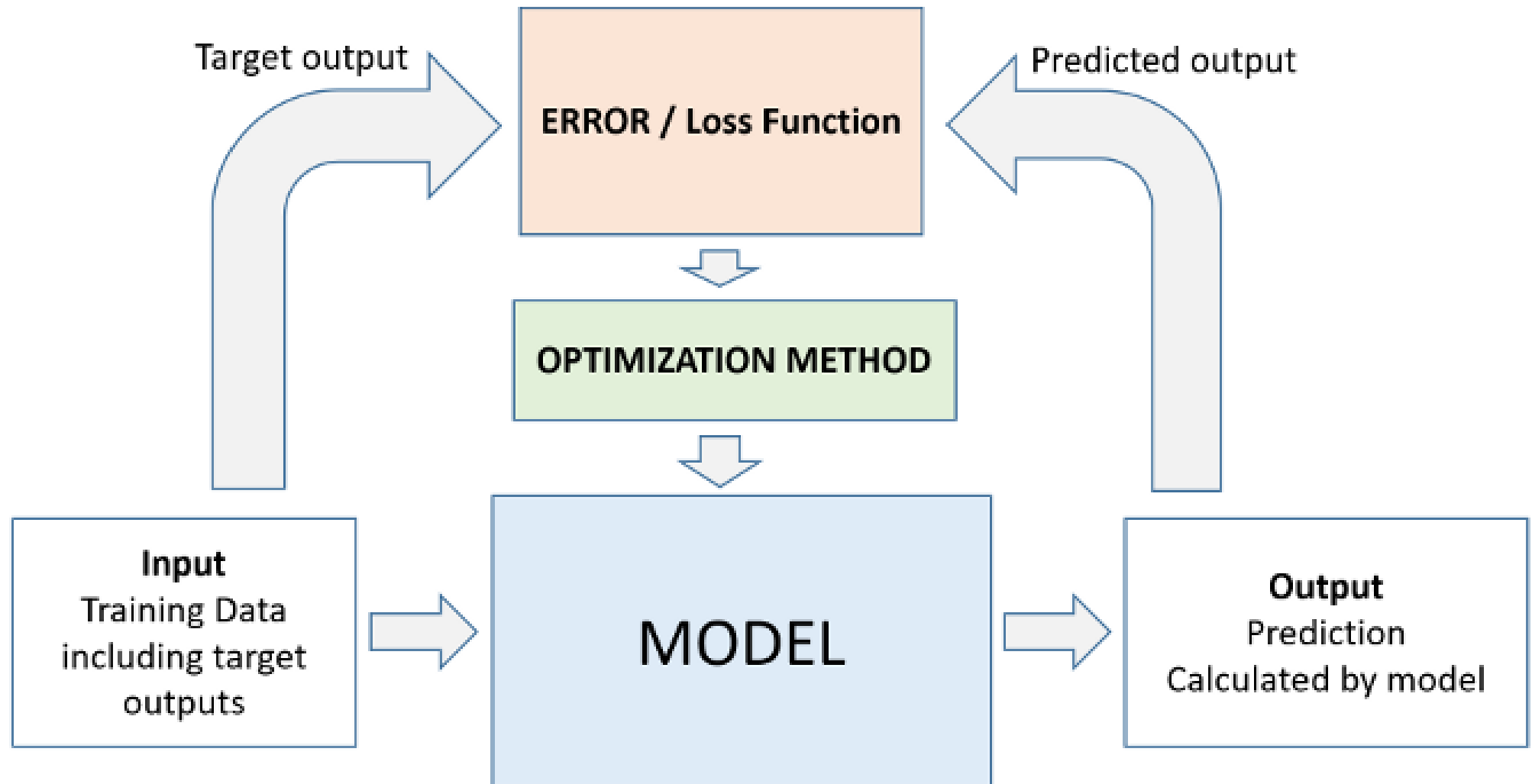


Figure 1-16. Model-based learning

Aprendizaje basado en modelos



Trabajar con datos reales

California Housing Prices

- Se puede utilizar el conjunto de datos **California Housing Prices** del **repositorio StatLib**. Este conjunto de datos se basa en los datos del censo de California de 1990.

<https://github.com/ageron/handson-ml/tree/master/datasets/housing>

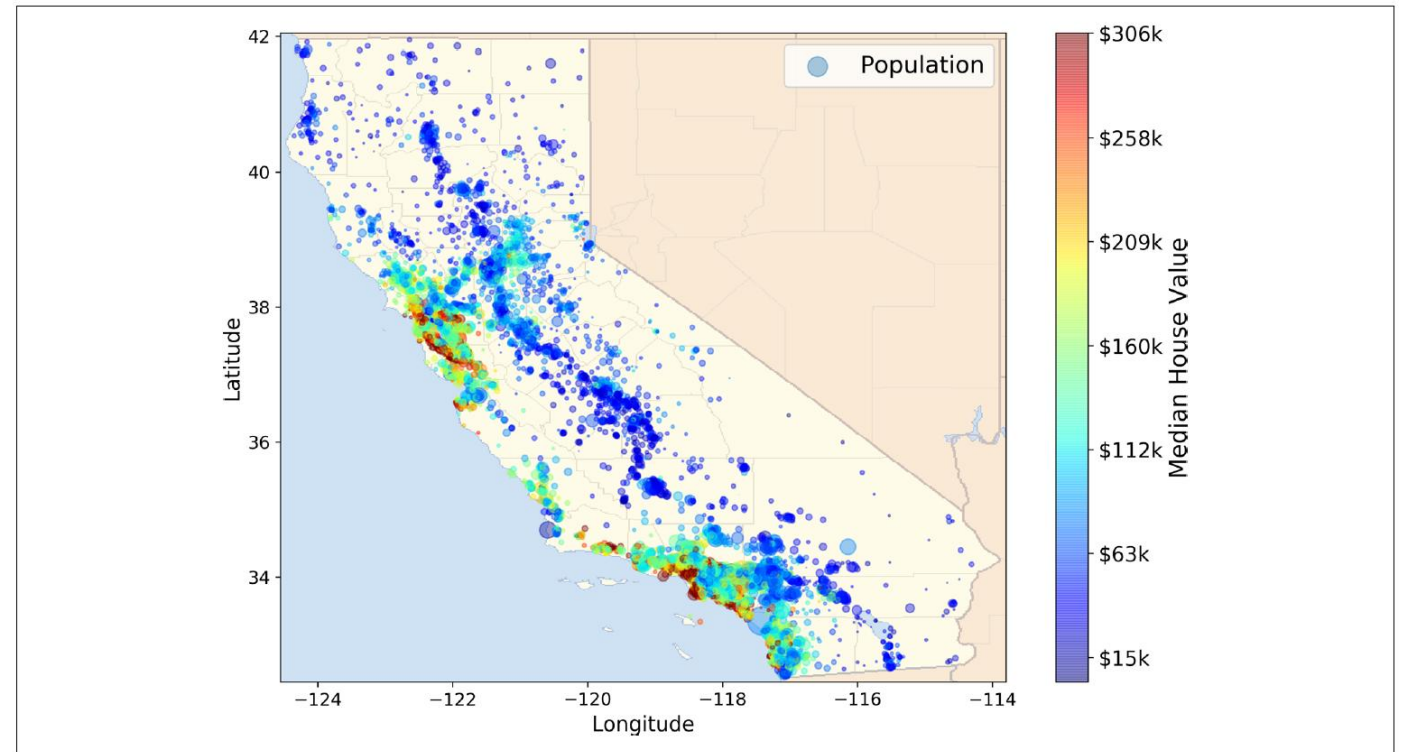
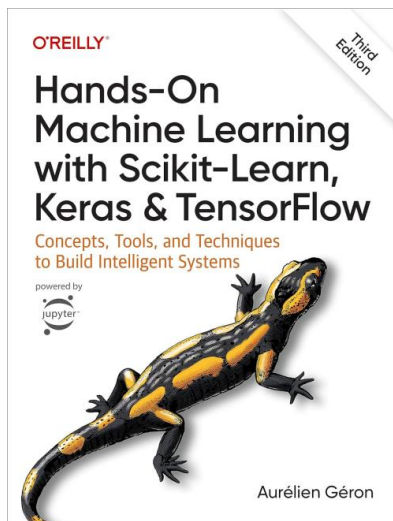


Figure 2-1. California housing prices

Trabajar con datos reales

California Housing Prices

- Estos datos incluyen parámetros como la población, la renta media y el precio medio de la vivienda para cada grupo de manzanas de California. Los grupos de manzanas son la unidad geográfica más pequeña para la que la Oficina del Censo de EE.UU. publica datos por muestreo. Los llamaremos "distritos" para abreviar.

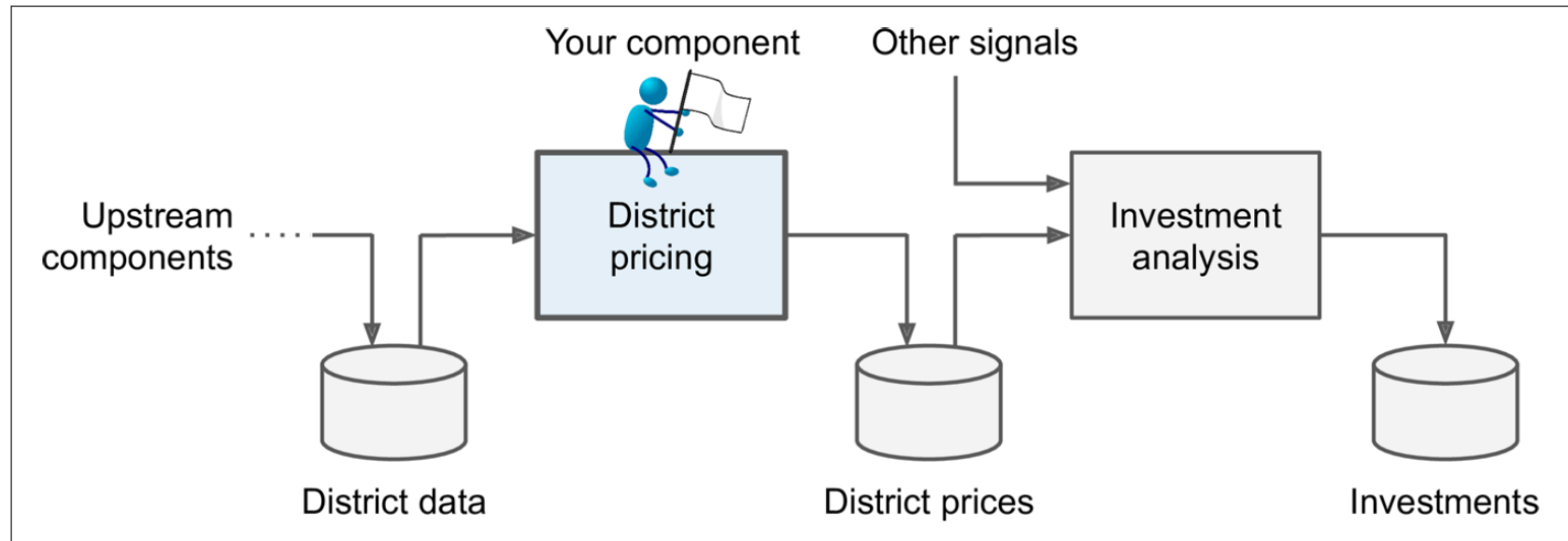


Figure 2-2. A Machine Learning pipeline for real estate investments

Trabajar con datos reales

- Se trata claramente de una tarea típica de **aprendizaje supervisado**, ya que se le dan ejemplos de entrenamiento etiquetados (cada instancia viene con la salida esperada, es decir, el precio medio de la vivienda del distrito).
- También es una **tarea de regresión** típica, ya que se le pide que prediga un valor. Más concretamente, se trata de un problema de regresión múltiple, ya que el sistema utilizará múltiples características para hacer una predicción (utilizará la población del distrito, la renta media, etc.).
- También se trata de un problema de regresión univariante, ya que sólo intentamos predecir un único valor para cada distrito.

Trabajar con datos reales

Seleccionar una medida de rendimiento

- Una medida de rendimiento típica **para los problemas de regresión es el error cuadrático medio (RMSE)**. Da una idea de cuánto error suele cometer el sistema en sus predicciones, con una ponderación mayor para los errores grandes.

$$\text{RMSE}(\mathbf{X}, h) = \sqrt{\frac{1}{m} \sum_{i=1}^m \left(h(\mathbf{x}^{(i)}) - y^{(i)} \right)^2}$$

- m es el número de instancias del conjunto de datos en el que se mide el RMSE.
- $\mathbf{x}^{(i)}$ es un vector de todos los valores de características (excluida la etiqueta) de la instancia i^{th} del conjunto de datos, e $y^{(i)}$ es su etiqueta (el valor de salida deseado para esa instancia).
- $\mathbf{X}$ es una matriz que contiene todos los valores de las características (excluidas las etiquetas) de todas las instancias del conjunto de datos. Hay una fila por instancia, y la fila i^{th} es igual a la transposición de $\mathbf{x}^{(i)}$, denominada $(\mathbf{x}^{(i)})^T$

Trabajar con datos reales

Seleccionar una medida de rendimiento

► Ejemplos de notación:

$$\mathbf{x}^{(1)} = \begin{pmatrix} -118.29 \\ 33.91 \\ 1,416 \\ 38,372 \end{pmatrix}$$

$$y^{(1)} = 156,400$$

$$\mathbf{X} = \begin{pmatrix} (\mathbf{x}^{(1)})^\top \\ (\mathbf{x}^{(2)})^\top \\ \vdots \\ (\mathbf{x}^{(1999)})^\top \\ (\mathbf{x}^{(2000)})^\top \end{pmatrix} = \begin{pmatrix} -118.29 & 33.91 & 1,416 & 38,372 \\ \vdots & \vdots & \vdots & \vdots \end{pmatrix}$$

Trabajar con datos reales

Seleccionar una medida de rendimiento

- h es la función de predicción del sistema, también llamada *hipótesis*. Cuando el sistema recibe un vector de características $\mathbf{x}^{(i)}$, genera un valor predicho $\hat{y}^{(i)} = h(\mathbf{x}^{(i)})$ para esa instancia.
- $\text{RMSE}(\mathbf{X}, h)$ es la función de coste medida en el conjunto de ejemplos utilizando su hipótesis h .
 - **Por ejemplo, si tu sistema predice que el precio medio de la vivienda en el primer distrito es de 158.400 \$, entonces $\hat{y}^{(1)} = h(\mathbf{x}^{(1)}) = 158.400$. El error de predicción para este distrito es $\hat{y}^{(1)} - y^{(1)} = 2.000$.**

$$y^{(1)} = 156,400$$

Diseño de Scikit-Learn (Prácticas)

- La **API de Scikit-Learn** está notablemente bien diseñada. Estos son los principales principios de diseño:
- Coherencia
 - Todos los objetos comparten una interfaz coherente y sencilla:
 - Estimadores
 - Cualquier objeto que pueda estimar algunos parámetros basándose en un conjunto de datos se denomina estimador (por ejemplo, un imputador es un estimador). La estimación en sí la realiza el método `fit()`, y sólo toma un conjunto de datos como parámetro (o dos para algoritmos de aprendizaje supervisado; el segundo conjunto de datos contiene las etiquetas). Cualquier otro parámetro necesario para guiar el proceso de estimación se considera un hiperparámetro (como la estrategia de imputación), y debe establecerse como una variable de instancia (generalmente a través de un parámetro constructor).

Diseño de Scikit-Learn (Prácticas)

► Transformadores

- Algunos estimadores (como un imputer) también pueden transformar un conjunto de datos; se denominan transformadores. Una vez más, la API es sencilla: la transformación la realiza el método `transform()` con el conjunto de datos a transformar como parámetro. Devuelve el conjunto de datos transformado. Esta transformación se basa generalmente en los parámetros aprendidos, como en el caso de un imputer. Todos los transformadores también tienen un método de conveniencia llamado `fit_transform()` que es equivalente a llamar a `fit()` y luego a `transform()` (pero a veces `fit_transform()` está optimizado y se ejecuta mucho más rápido).

► Predictores

- Algunos estimadores, dado un conjunto de datos, son capaces de hacer predicciones; se denominan predictores. Por ejemplo, el modelo `LinearRegression` es un predictor: dado el PIB per cápita de un país, predice la satisfacción con la vida. Un predictor tiene un método `predict()` que toma un conjunto de datos de nuevas instancias y devuelve un conjunto de datos de predicciones correspondientes. También tiene un método `score()` que mide la calidad de las predicciones, dado un conjunto de pruebas (y las etiquetas correspondientes, en el caso de los algoritmos de aprendizaje supervisado).

Diseño de Scikit-Learn (Prácticas)

■ Inspección

- Todos los hiperparámetros del estimador son accesibles directamente a través de variables de instancia públicas (por ejemplo, `imputer.strategy`), y todos los parámetros aprendidos del estimador son accesibles a través de variables de instancia públicas con un sufijo de guión bajo (por ejemplo, `imputer.statistics_`).

■ No proliferación de clases

- Los conjuntos de datos se representan como matrices NumPy o matrices dispersas SciPy, en lugar de clases caseras. Los hiperparámetros son cadenas o números normales de Python.

■ Composición

- Los bloques de construcción existentes se reutilizan en la medida de lo posible. Por ejemplo, es fácil crear un estimador Pipeline a partir de una secuencia arbitraria de transformadores seguida de un estimador final.

■ Valores por defecto sensatos

- Scikit-Learn proporciona valores predeterminados razonables para la mayoría de los parámetros, lo que facilita la creación rápida de un sistema de trabajo básico.



Aprendizaje supervisado

Introducción

Evaluación de modelos

Aprendizaje basado en instancias

Modelo Naïve Bayes

Regresión Lineal

Modelos basados en árboles de decisión

Modelos multclasificadores y de
descomposición

Seleccionar una medida de rendimiento

- Aunque el RMSE es generalmente la medida de rendimiento preferida para las tareas de regresión, en algunos contextos puede que prefiera utilizar otra función.
- Por ejemplo, supongamos que hay muchos distritos atípicos. En ese caso, puede considerar utilizar el error medio absoluto (MAE, también llamado desviación media absoluta).

$$\text{MAE}(\mathbf{X}, h) = \frac{1}{m} \sum_{i=1}^m |h(\mathbf{x}^{(i)}) - y^{(i)}|$$

- Tanto el RMSE como el MAE son formas de medir la distancia entre dos vectores: el vector de predicciones y el vector de valores objetivo. Existen varias medidas de distancia, o *normas*:
 - Norma euclidiana, ℓ_2 , anotada $\|\cdot\|_2$ (o simplemente $\|\cdot\|$).
 - Manhattan, ℓ_1 norma, anotada $\|\cdot\|_1$
 - ℓ_k norma de un vector $\mathbf{v}$, anotada $\|\mathbf{v}\|_k = (|v_0|^k + |v_1|^k + \dots + |v_n|^k)^{1/k}$
 - ℓ_0 da el número de elementos distintos de cero en el vector, y ℓ_∞ da el valor absoluto máximo en el vector.

Matriz de confusión

- La matriz de confusión proporciona mucha información, pero a veces se prefiere una métrica más concisa. Un parámetro interesante es la **exactitud de las predicciones positivas**, que se denomina *precisión* del clasificador.

$$\text{precision} = \frac{TP}{TP + FP}$$

- TP es el número de verdaderos positivos, y FP es el número de falsos positivos.
- Una forma trivial de tener una precisión perfecta es hacer una única predicción positiva y asegurarse de que es correcta (precisión = $1/1 = 100\%$). Pero esto no sería muy útil, ya que el clasificador ignoraría todos los casos positivos excepto uno.
- Por eso, la precisión se suele utilizar junto con otra métrica llamada recall, también llamada **sensibilidad o tasa de verdaderos positivos** (TPR):

es la proporción de casos positivos que el clasificador detecta correctamente:

$$\text{recall} = \frac{TP}{TP + FN}$$

Precisión y recuperación

- A menudo es conveniente combinar la precisión y la recuperación en una única métrica denominada puntuación F_1 , en particular si se necesita una forma sencilla de comparar dos clasificadores.
- La puntuación F_1 **es la media armónica de precisión y sensibilidad (recall)**. Mientras que la media normal trata todos los valores por igual, la media armónica da mucha más importancia a los valores bajos.
- **Como resultado, el clasificador sólo obtendrá una puntuación F_1 alta si tanto el recuerdo como la precisión son altos.**

$$F_1 = \frac{2}{\frac{1}{\text{precision}} + \frac{1}{\text{recall}}} = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} = \frac{TP}{TP + \frac{FN + FP}{2}}$$

$$\text{precision} = \frac{TP}{TP + FP}$$

$$\text{recall} = \frac{TP}{TP + FN}$$

Compromiso precisión/recuperación

- Para entender este equilibrio, veamos cómo toma el *clasificador* sus decisiones de clasificación.
- Para cada instancia, calcula una puntuación basada en una *función de decisión*. Si esa puntuación es superior a un umbral, asigna la instancia a la clase positiva; en caso contrario, la asigna a la clase negativa.

Matriz de confusión: Predicción del dígito 5

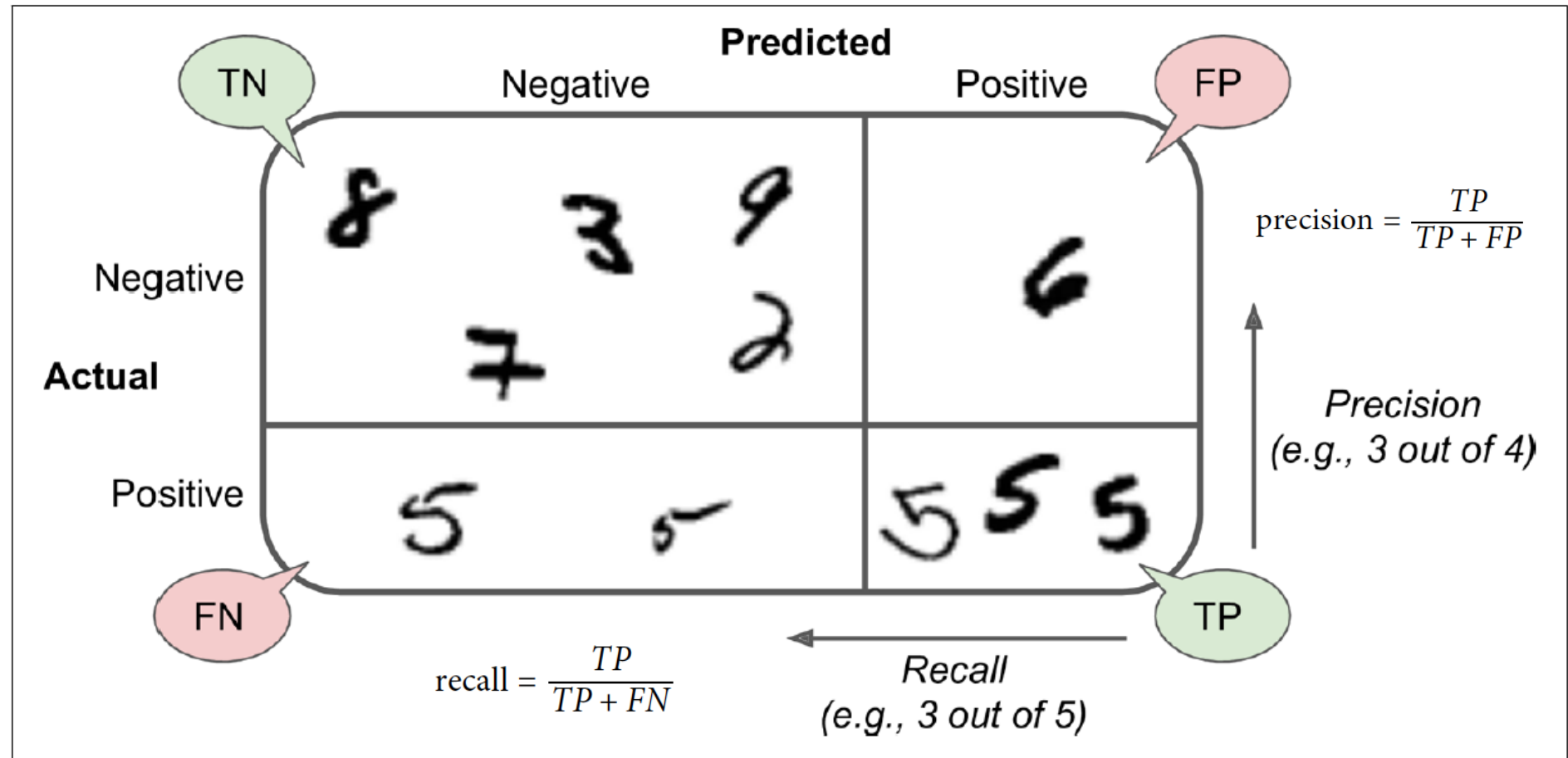


Figure 3-2. An illustrated confusion matrix shows examples of true negatives (top left), false positives (top right), false negatives (lower left), and true positives (lower right)

Compromiso precisión/recuperación

- La figura de la siguiente diapositiva muestra algunos dígitos situados desde la puntuación más baja a la izquierda hasta la más alta a la derecha. Supongamos que el umbral de decisión se sitúa en la flecha central (entre los dos 5): encontrará 4 verdaderos positivos (5 reales) a la derecha de ese umbral, y 1 falso positivo (en realidad un 6). Por tanto, con ese umbral, la precisión es del 80% (4 de 5). Pero de los 6 5 reales, el clasificador sólo detecta 4, por lo que la recuperación es del 67% (4 de 6). Si se aumenta el umbral (moviéndolo hacia la flecha de la derecha), el falso positivo (el 6) se convierte en un verdadero negativo, con lo que aumenta la precisión (hasta el 100% en este caso), pero un verdadero positivo se convierte en un falso negativo, con lo que la recuperación disminuye hasta el 50%. Por el contrario, si se reduce el umbral, aumenta la recuperación y disminuye la precisión.

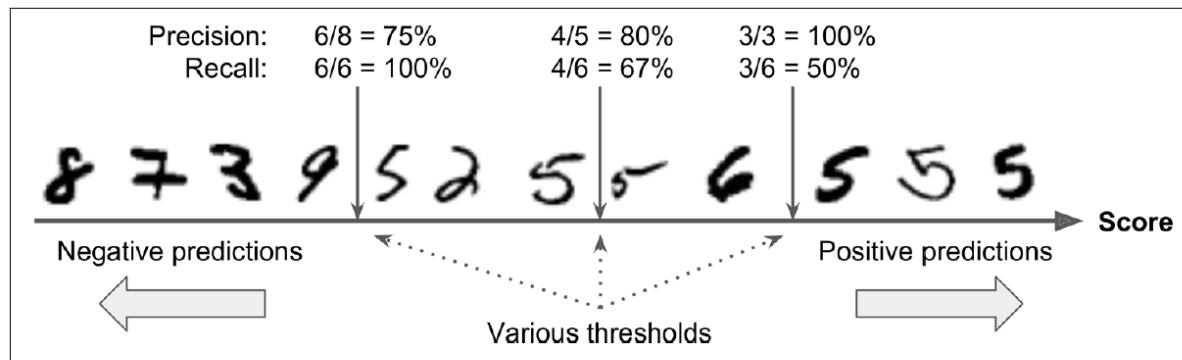


Figure 3-3. In this precision/recall trade-off, images are ranked by their classifier score, and those above the chosen decision threshold are considered positive; the higher the threshold, the lower the recall, but (in general) the higher the precision

$$\text{recall} = \frac{TP}{TP + FN}$$

$$\text{precision} = \frac{TP}{TP + FP}$$

Compromiso precisión/recuperación

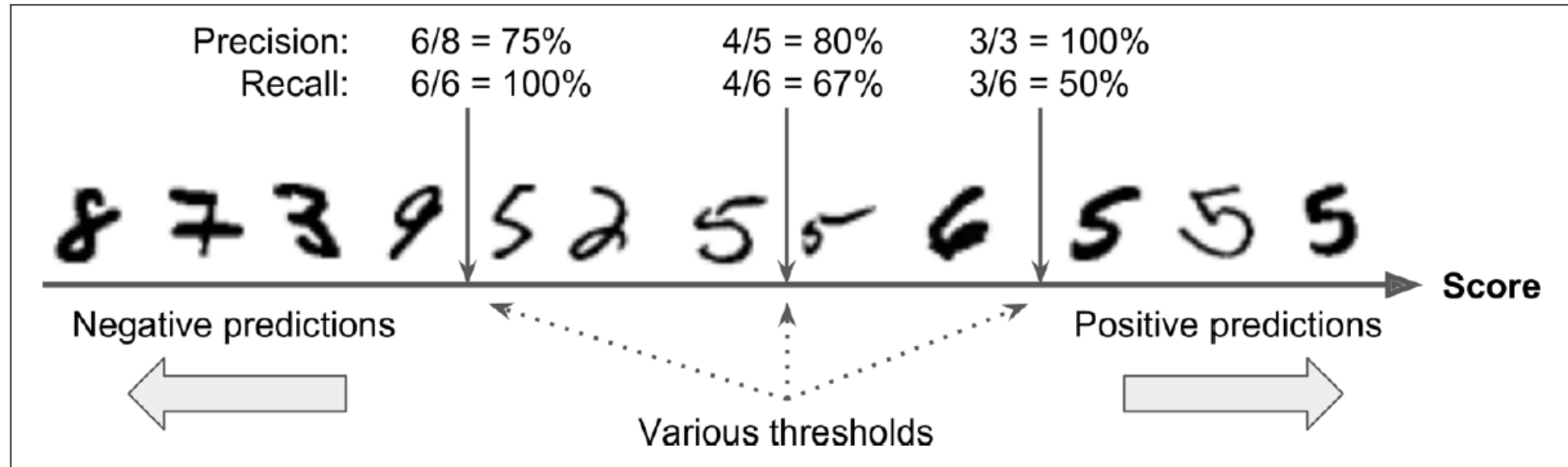


Figure 3-3. In this precision/recall trade-off, images are ranked by their classifier score, and those above the chosen decision threshold are considered positive; the higher the threshold, the lower the recall, but (in general) the higher the precision

$$\text{precision} = \frac{TP}{TP + FP}$$

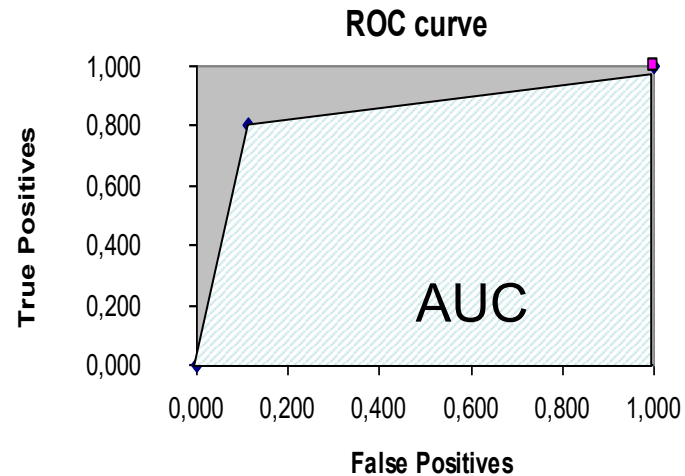
$$\text{recall} = \frac{TP}{TP + FN}$$

La curva ROC

- La curva *receiver operating characteristic* (ROC) es otra herramienta común utilizada con clasificadores binarios.
- Es muy similar a la curva de precisión/recuperación, pero en lugar de representar la precisión frente a la recuperación, la curva ROC representa la *tasa de verdaderos positivos* (otro nombre para la recuperación) frente a la *tasa de falsos positivos* (FPR).
- El FPR es la proporción de instancias negativas que se clasifican incorrectamente como positivas.
- Es igual a $1 - \text{tasa de verdaderos negativos (TNR)}$, que es la proporción de instancias negativas que se clasifican correctamente como negativas.
- El TNR también se denomina *especificidad*.
- Por lo tanto, la curva ROC traza la *sensibilidad* (recuerdo) frente a $1 - \text{especificidad}$.

La curva ROC

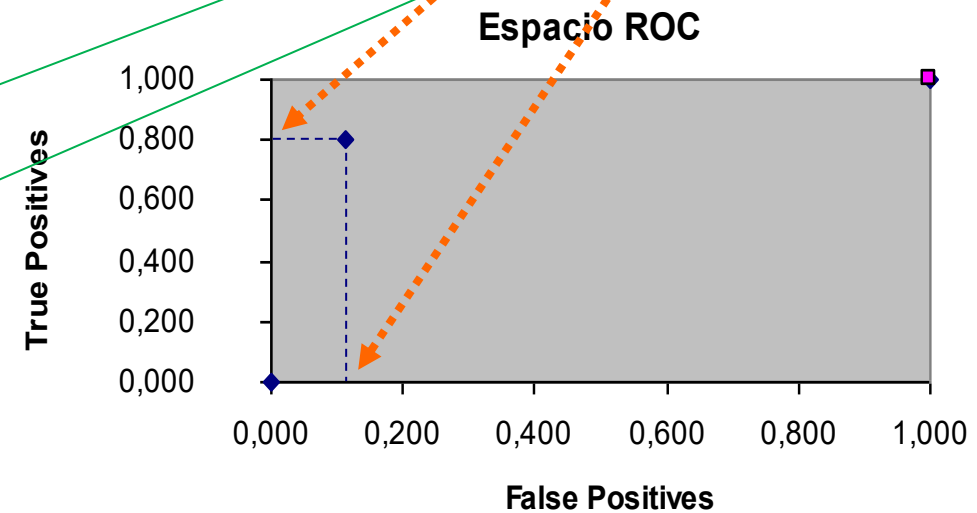
Por lo tanto, la curva ROC traza la *sensibilidad* (recuerdo) frente a $1 - \text{especificidad}$.



Pred

	PP	NP
PC	0,8	0,121
NC	0,2	0,879

$$\text{AUC} = \frac{1 + \text{TP}_{\text{rate}} - \text{FP}_{\text{rate}}}{2}$$



Clasificación multiclase

- Mientras que los clasificadores binarios distinguen entre dos clases, *los clasificadores multiclase* (también llamados *clasificadores multinomiales*) pueden distinguir entre más de dos clases.
- Algunos algoritmos (son capaces de manejar múltiples clases de forma nativa.
- Otros (como la regresión logística o los clasificadores de máquina de vectores de soporte) son clasificadores estrictamente binarios. Sin embargo, hay varias estrategias que puede utilizar para realizar la clasificación multiclase con varios clasificadores binarios.
- Una forma de crear un sistema que pueda clasificar las imágenes de dígitos en 10 clases (de 0 a 9) es entrenar 10 clasificadores binarios, uno para cada dígito (un detector 0, un detector 1, un detector 2, etc.).
- A continuación, cuando se desea clasificar una imagen, se obtiene la puntuación de decisión de cada clasificador para esa imagen y se selecciona la clase cuyo clasificador arroja la puntuación más alta.
- Es lo que se denomina la estrategia *uno contra todos* (OvA) (también llamada *uno contra el resto* (OvR)).

Clasificación multiclase

- Otra estrategia consiste en entrenar un clasificador binario para cada par de dígitos: uno para distinguir 0s y 1s, otro para distinguir 0s y 2s, otro para 1s y 2s, y así sucesivamente.
- Esto se denomina estrategia *uno contra uno* (OvO).
- Si hay N clases, hay que entrenar $N \times (N - 1) / 2$ clasificadores. Para el problema MNIST, esto significa entrenar 45 clasificadores binarios. Cuando se quiere clasificar una imagen, hay que pasarla por los 45 clasificadores y ver qué clase gana más duelos.
- La principal ventaja de OvO es que cada clasificador sólo necesita ser entrenado en la parte del conjunto de entrenamiento correspondiente a las dos clases que debe distinguir.
- Algunos algoritmos (como los clasificadores de la máquina de vectores de soporte) no se adaptan bien al tamaño del conjunto de entrenamiento. Para estos algoritmos se prefiere OvO porque es más rápido entrenar muchos clasificadores en conjuntos de entrenamiento pequeños que entrenar pocos clasificadores en conjuntos de entrenamiento grandes.
- Sin embargo, para la mayoría de los algoritmos de clasificación binaria se prefiere OvA.



Aprendizaje supervisado

Introducción

Evaluación de modelos

Aprendizaje basado en instancias

Regresión Lineal

Modelo Naïve Bayes

Modelos basados en árboles de decisión

Modelos multclasificadores y de
descomposición

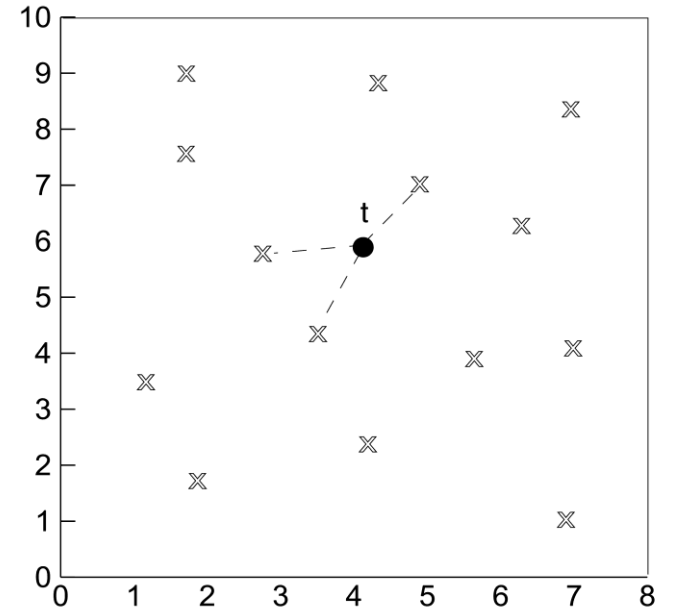
Aprendizaje basado en instancias: kNN

Regla del vecino más próximo (1-NN)

- Si tenemos m instancias $\{x_1, \dots, x_m\}$ en nuestra base de datos, para clasificar una nueva instancia x' se hará lo siguiente:

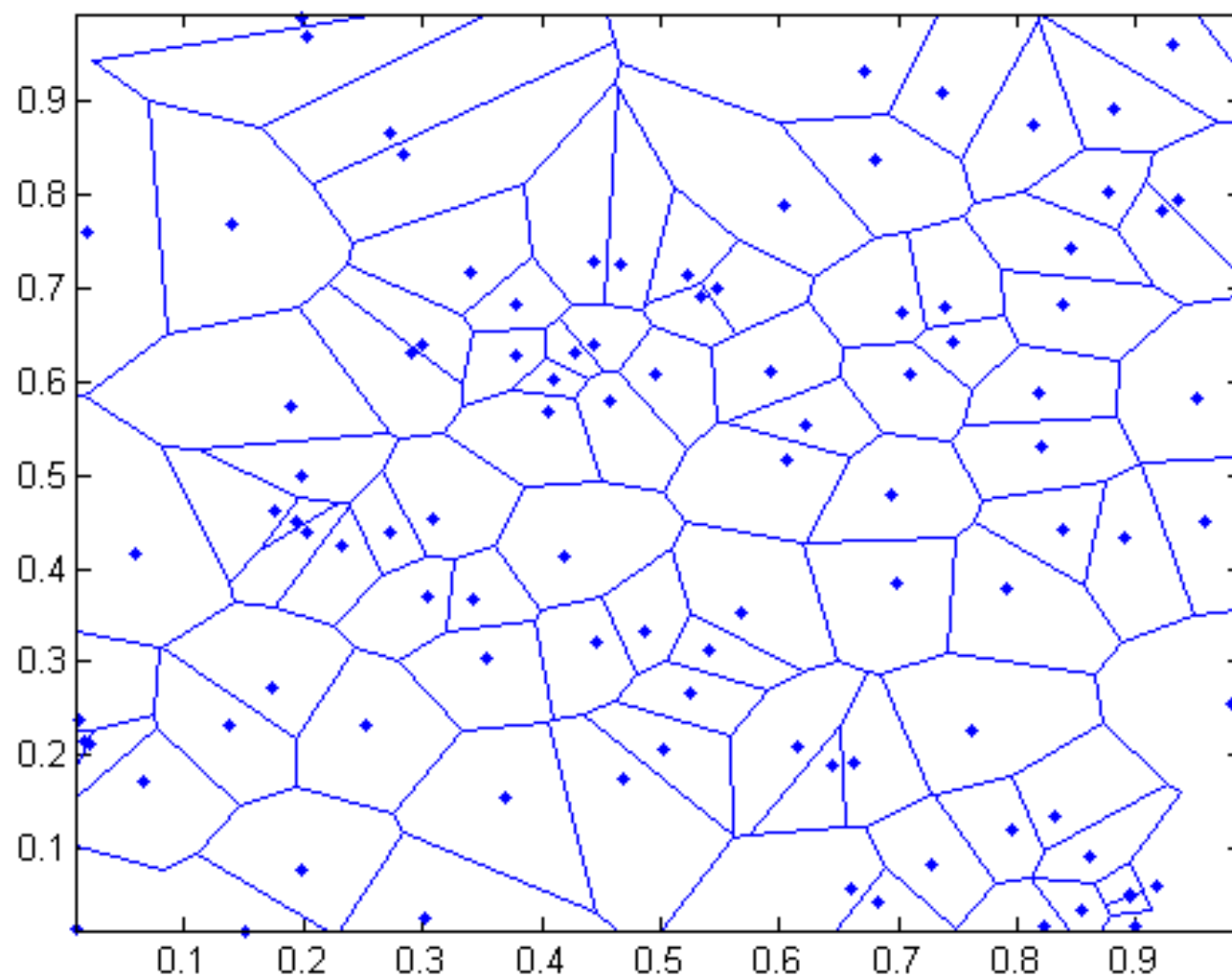
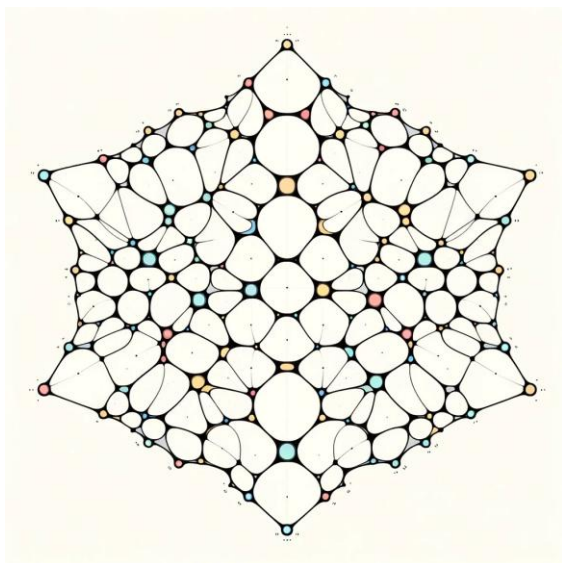
- $c_{min} = \text{clase}(x_1)$
- $d_{min} = d(x_1, x')$
- Para $i=2$ a m hacer
 $d = d(x_i, x')$
 Si $(d < d_{min})$
 Entonces $c_{min} = \text{clase}(x_i)$, $d_{min} = d$
- Devuelve c_{min} como clase predicha para x'

- $d(-, -)$ es una función de distancia (norma)
- En el caso de las variables nominales, se suele utilizar Hamming:



$$d_h(a, b) = \begin{cases} 0, & \text{si } a = b \\ 1, & \text{si } a \neq b \end{cases}$$

Aprendizaje basado en instancias: kNN



Aprendizaje basado en instancias: kNN

Clasificador Del Vecino Más Cercano (k-NN) Problema en las fronteras

Ejemplo: Diseño de un Clasificador para *Iris*

- ❑ Problema simple muy conocido: *clasificación de lirios*.
- ❑ Tres clases de lirios: *setosa*, *versicolor* y *virginica*.
- ❑ Cuatro atributos: *longitud* y *anchura* de *pétalo* y *sépalo*, respectivamente.
- ❑ 150 ejemplos, 50 de cada clase.
- ❑ Disponible en <http://www.ics.uci.edu/~mlern/MLRepository.html>

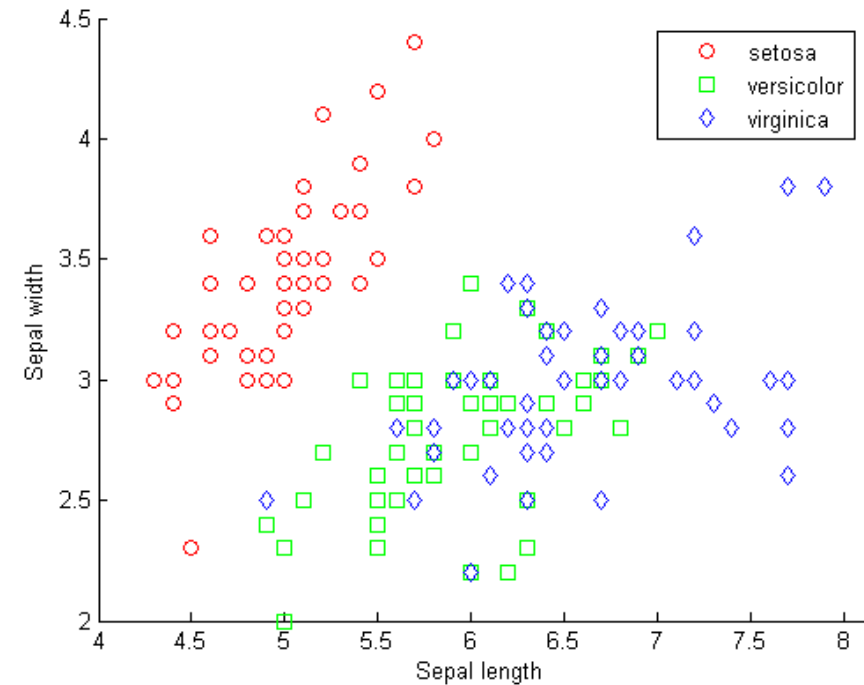
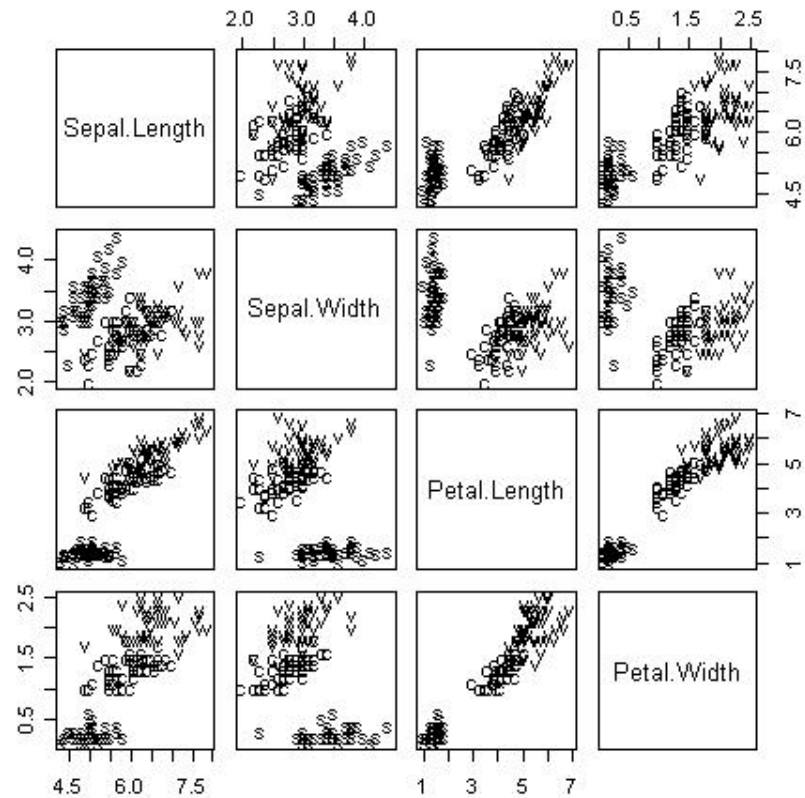


Aprendizaje basado en instancias: kNN

	Sepal length X_1	Sepal width X_2	Petal length X_3	Petal width X_4	Class X_5
$\mathbf{x}_1$	5.9	3.0	4.2	1.5	Iris-versicolor
$\mathbf{x}_2$	6.9	3.1	4.9	1.5	Iris-versicolor
$\mathbf{x}_3$	6.6	2.9	4.6	1.3	Iris-versicolor
$\mathbf{x}_4$	4.6	3.2	1.4	0.2	Iris-setosa
$\mathbf{x}_5$	6.0	2.2	4.0	1.0	Iris-versicolor
$\mathbf{x}_6$	4.7	3.2	1.3	0.2	Iris-setosa
$\mathbf{x}_7$	6.5	3.0	5.8	2.2	Iris-virginica
$\mathbf{x}_8$	5.8	2.7	5.1	1.9	Iris-virginica
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
$\mathbf{x}_{149}$	7.7	3.8	6.7	2.2	Iris-virginica
$\mathbf{x}_{150}$	5.1	3.4	1.5	0.2	Iris-setosa

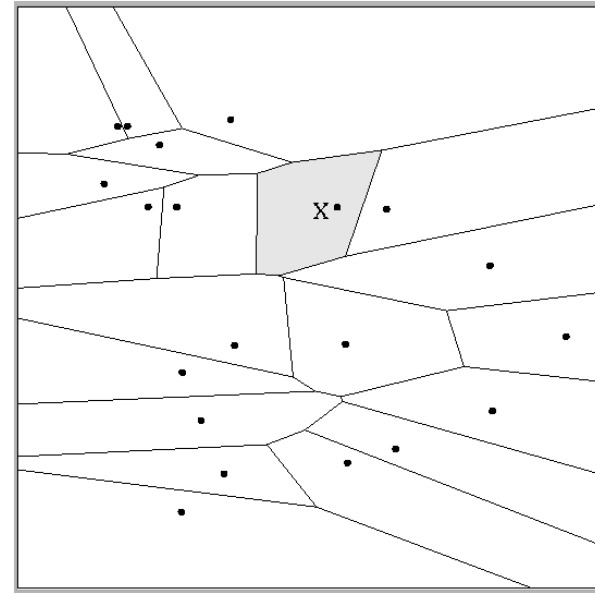
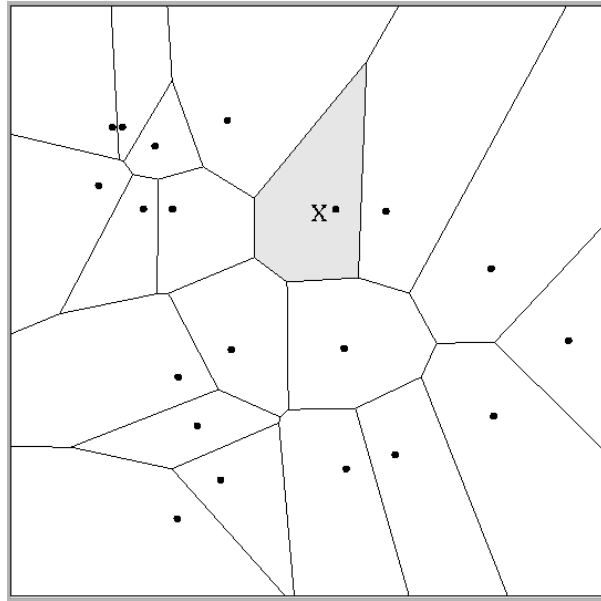
Aprendizaje basado en instancias: kNN

setosa, versicolor (C) y virginica



Aprendizaje basado en instancias: kNN

Ponderación de características



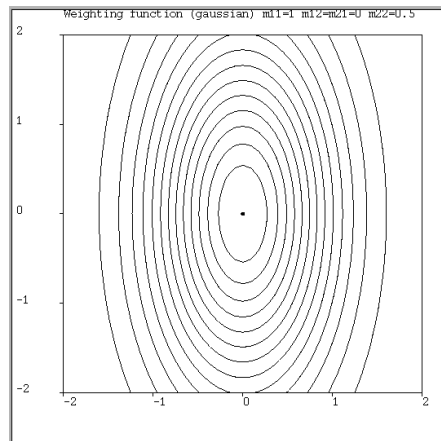
$$Dist(\mathbf{x}_i, \mathbf{x}_j) = (x_{i1} - x_{j1})^2 + (x_{i2} - x_{j2})^2 \quad Dist(\mathbf{x}_i, \mathbf{x}_j) = (x_{i1} - x_{j1})^2 + (3x_{i2} - 3x_{j2})^2$$

Las escalas relativas en la métrica de la distancia afectan a las formas de las regiones.

Aprendizaje basado en instancias: kNN

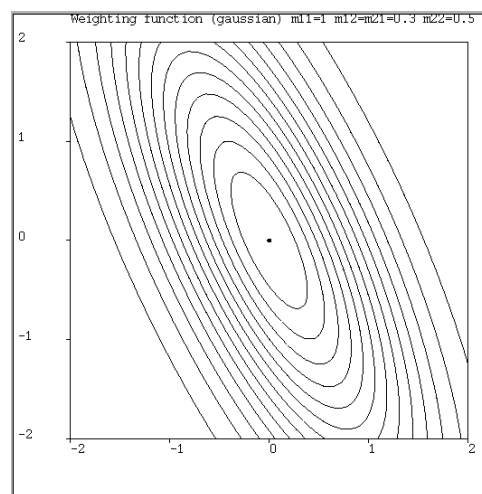
Métricas de distancia interesantes

**Euclideo escalado
(L)₂**

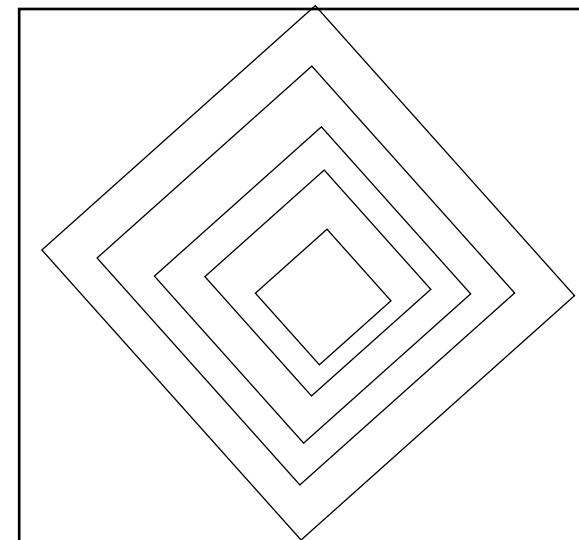


$$D(x, x') = \sqrt{\sum_i \sigma_i^2 (x_i - x'_i)^2}$$
$$D(x, x') = \sqrt{(x - x')^T \Sigma (x - x')}$$
$$\Sigma = \begin{bmatrix} \sigma_1^2 & 0 & \dots & 0 \\ 0 & \sigma_2^2 & \dots & 0 \\ \dots & \dots & \dots & \dots \\ 0 & 0 & \dots & \sigma_N^2 \end{bmatrix}$$

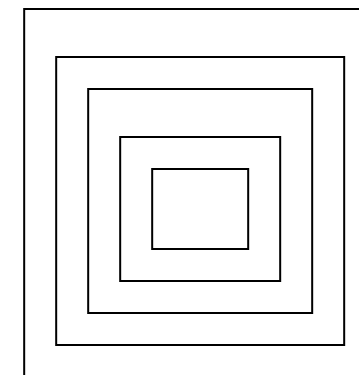
Mahalanobis



L₁ norma (absoluta)



**Norma *L*_{infinity}
(max)**



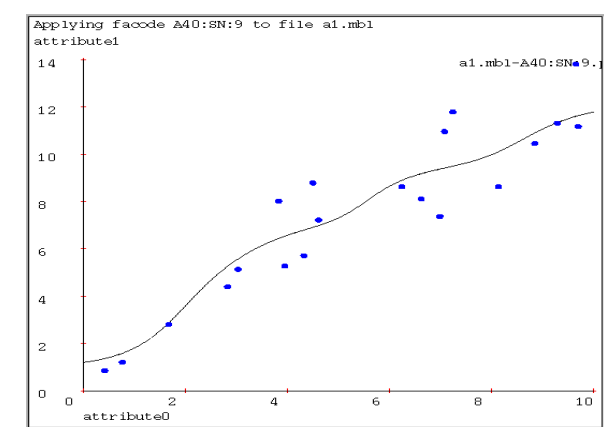
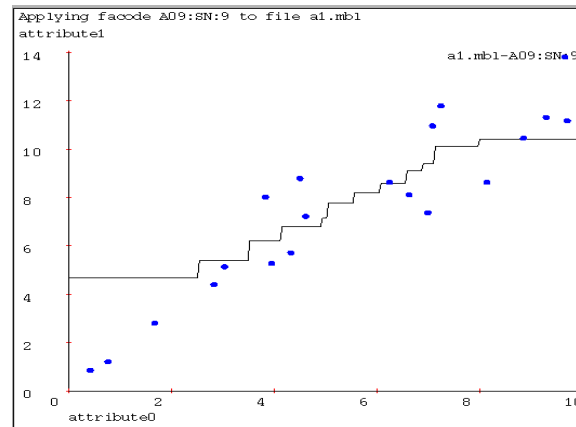
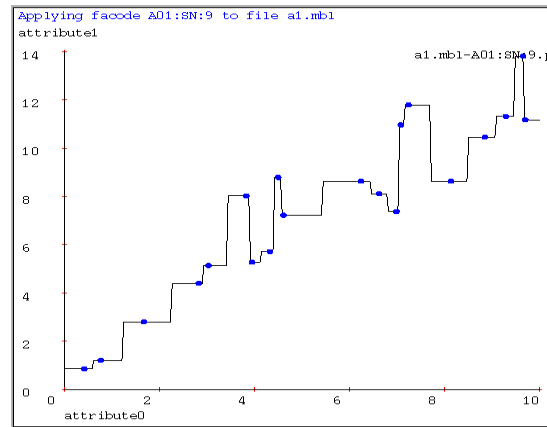
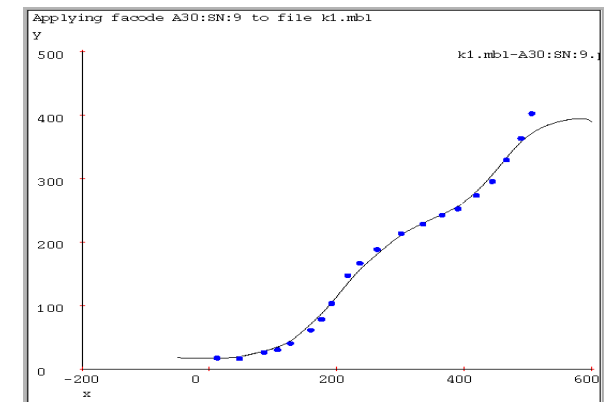
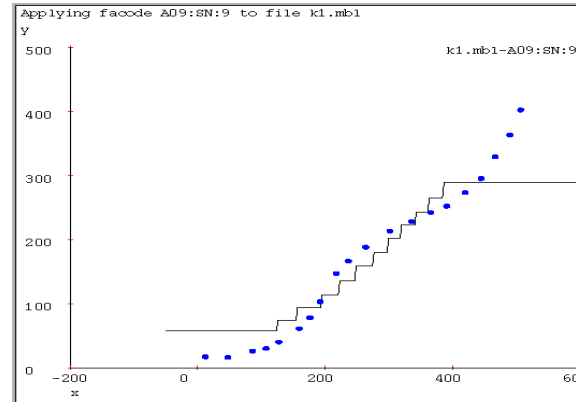
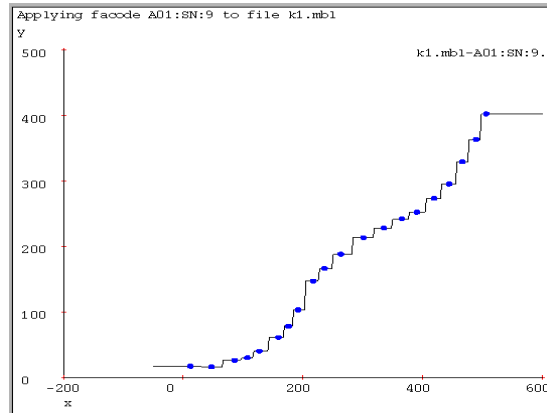
Aprendizaje basado en instancias: kNN

kNN en regresión

9-NN: Predice según la media del vecindario

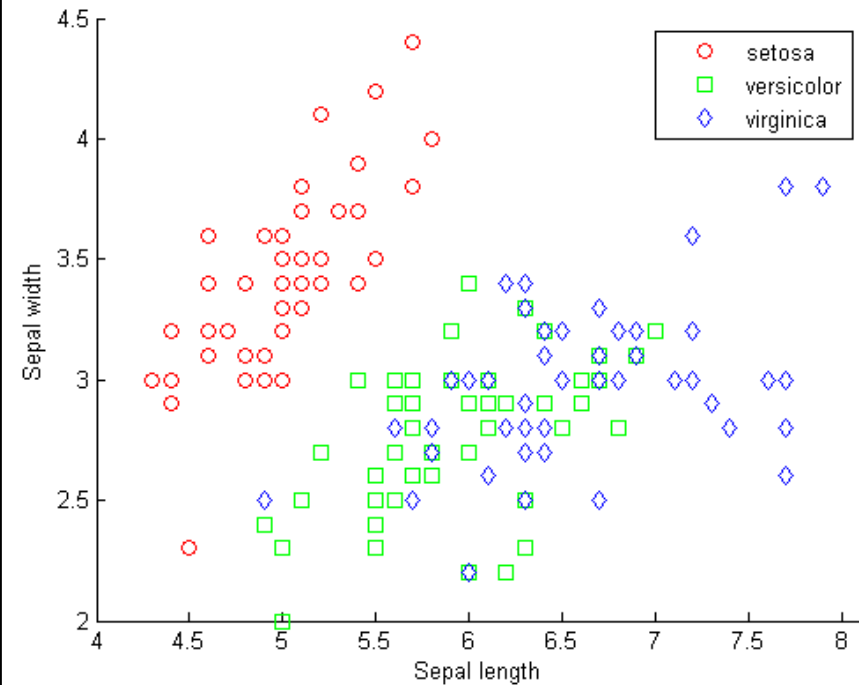
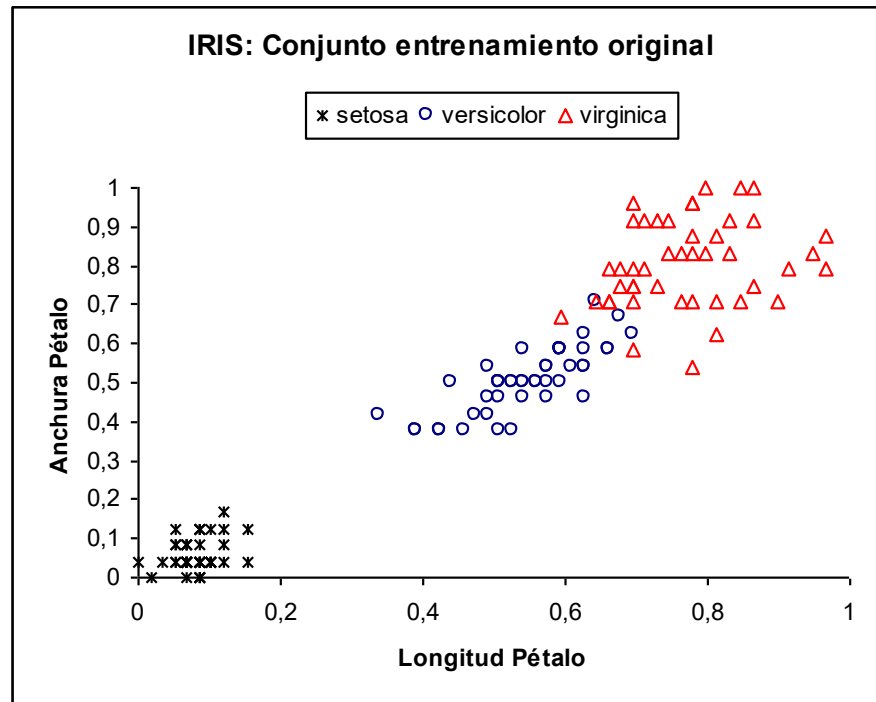
1-NN

Núcleo



Aprendizaje basado en instancias: kNN

setosa, versicolor (C) y virginica





Aprendizaje supervisado

Introducción

Evaluación de modelos

Aprendizaje basado en instancias

Regresión Lineal

Modelo Naïve Bayes

Modelos basados en árboles de decisión

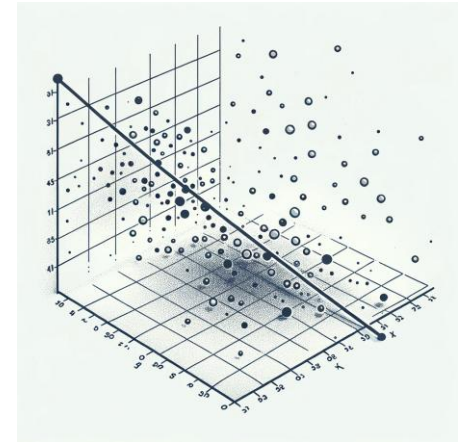
Modelos multclasificadores y de
descomposición

Regresión lineal

- Un modelo lineal realiza una predicción simplemente calculando una suma ponderada de las características de entrada, más una constante denominada *término de sesgo*.

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

- $\hat{y}$ es el valor previsto.
- n es el número de características.
- x_i es el valor de la característica i^{th} .
- θ_j es el parámetro j del modelo th , que incluye el término de sesgo θ_0 y las ponderaciones de las características $\theta_1, \theta_2, \dots, \theta_n$.



Regresión lineal

- Esto puede escribirse de forma mucho más concisa utilizando una forma vectorizada:

$$\hat{y} = h_{\theta}(\mathbf{x}) = \theta \cdot \mathbf{x}$$

- h_{θ} es la función de hipótesis, utilizando los parámetros del modelo θ .
- θ es el vector de parámetros del modelo, que contiene el término de sesgo θ_0 y las ponderaciones de las características θ_1 a θ_n .
- $\mathbf{x}$ es el vector de características de la instancia, que contiene de x_0 a x_n , siendo x_0 siempre igual a 1.
- $\theta \cdot \mathbf{x}$ es el producto punto de los vectores θ y $\mathbf{x}$, que es igual a
$$\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n.$$
- En el aprendizaje automático, los vectores suelen representarse como vectores columna, que son matrices 2D con una sola columna. Si θ y $\mathbf{x}$ son vectores columna, la predicción es $\hat{y} = \theta^T \mathbf{x}$.
- Es la misma predicción, salvo que ahora se representa como una matriz unicelda en lugar de un valor escalar.

Regresión lineal

- ¿Cómo se entrena? Recordemos que entrenar un modelo significa ajustar sus parámetros para que se adapte lo mejor posible al conjunto de entrenamiento.
- Para entrenar un modelo de regresión lineal, necesitamos encontrar el valor de θ que minimice el RMSE. En la práctica, es más sencillo minimizar el error cuadrático medio (MSE) que el RMSE, y conduce al mismo resultado.
- **Los algoritmos de aprendizaje suelen optimizar una función de pérdida diferente durante el entrenamiento que la medida de rendimiento utilizada para evaluar el modelo final.** Esto se debe generalmente a que la función es más fácil de optimizar y/o porque tiene términos adicionales necesarios sólo durante el entrenamiento (por ejemplo, para la regularización). Una buena *pérdida* para entrenamiento es fácil de optimizar y está fuertemente correlacionada con la métrica.
- El MSE de una hipótesis de regresión lineal h_θ sobre un conjunto de entrenamiento $\mathbf{X}$ se calcula utilizando:

$$\text{MSE}(\theta)$$

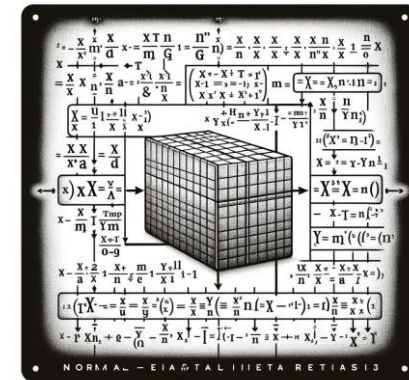
$$\text{MSE}(\mathbf{X}, h_\theta) = \frac{1}{m} \sum_{i=1}^m (\theta^\top \mathbf{x}^{(i)} - y^{(i)})^2$$

La ecuación normal

- Para encontrar el valor de θ que minimiza el MSE, existe una solución de forma cerrada, es decir, una ecuación matemática que da el resultado directamente.
- Esto se denomina *ecuación Normal*.

$$\hat{\theta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

- $\hat{\theta}$ es el valor de θ que minimiza la función de coste.
- $\mathbf{y}$ es el vector de valores objetivo que contiene $y^{(1)}$ a $y^{(m)}$.



- **Esto implica el cálculo de matrices inversas, lo que no es fácil. Solemos calcular pseudoinversas.**

La ecuación normal

- La pseudoinversa propiamente dicha se calcula mediante una técnica estándar de factorización de matrices denominada **descomposición de valores singulares (SVD)** que puede descomponer la matriz $\mathbf{X}$ del conjunto de entrenamiento en la multiplicación de 3 matrices $\mathbf{U} \mathbf{\Sigma} \mathbf{V}^T$.
- La pseudoinversa se calcula como $\mathbf{X}^+ = \mathbf{V} \mathbf{\Sigma}^+ \mathbf{U}^T$. Para calcular la matriz $\mathbf{\Sigma}^+$, el algoritmo toma $\mathbf{\Sigma}$ y pone a cero todos los valores inferiores a un valor umbral minúsculo, luego sustituye todos los valores distintos de cero por su inversa y, por último, transpone la matriz resultante.
- Este enfoque es más eficaz que el cálculo de la ecuación Normal, además de que maneja muy bien los casos extremos: de hecho, la ecuación Normal puede no funcionar si la matriz $\mathbf{X}^T \mathbf{X}$ no es invertible (es decir, singular), como cuando $m < n$ o cuando algunas características son redundantes.
- La pseudoinversa siempre está definida.

Complejidad computacional

- **La ecuación Normal** calcula la inversa de $\mathbf{X}^T \mathbf{X}$, que es una matriz $(n + 1) \times (n + 1)$.
- La complejidad computacional de invertir una matriz de este tipo suele ser de $O(n^{2.4})$ a $O(n^3)$, dependiendo de la implementación.
- **El enfoque SVD (descomposición en valores singulares, SVD) utilizado por la clase LinearRegression de Scikit-Learn es aproximadamente $O(n^2)$. Si se duplica el número de características, el tiempo de cálculo se multiplica aproximadamente por 4.**
- **Tanto la ecuación Normal como el enfoque SVD se vuelven muy lentos cuando el número de características aumenta (por ejemplo, 100.000). Por el lado positivo, ambos son lineales con respecto al número de instancias en el conjunto de entrenamiento (son $O(m)$), por lo que manejan grandes conjuntos de entrenamiento de manera eficiente, siempre que quepan en la memoria.**
- Las predicciones son muy rápidas: la complejidad computacional es lineal con respecto tanto al número de instancias sobre las que se quieren hacer predicciones como al número de características.

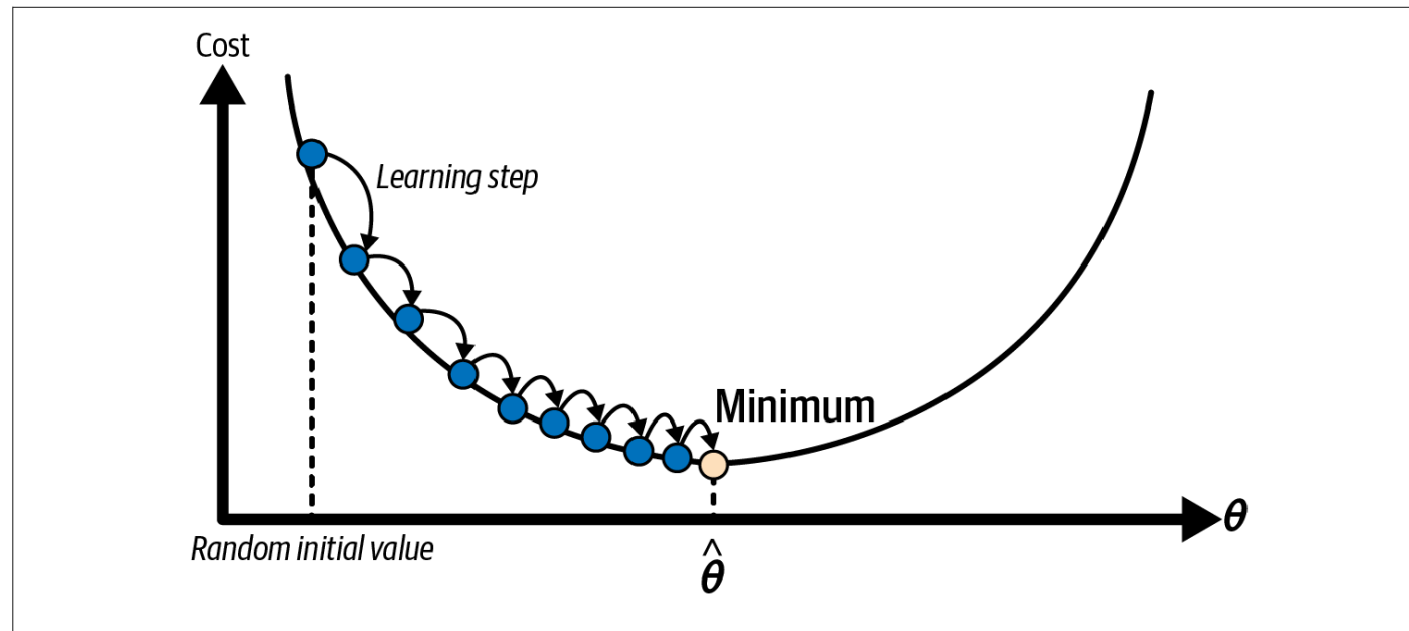
Descenso de gradiente

- **El descenso de gradiente es un algoritmo de optimización genérico capaz de encontrar soluciones óptimas a una amplia gama de problemas.** La idea general del descenso de gradiente es ajustar los parámetros de forma iterativa para minimizar una función de coste.
- Supongamos que estás perdido en las montañas, en medio de una densa niebla, y sólo puedes sentir la pendiente del suelo bajo tus pies. Una buena estrategia para llegar rápidamente al fondo del valle es ir cuesta abajo en la dirección de la pendiente más pronunciada.

Esto es exactamente lo que hace el descenso por gradiente: mide el gradiente local de la función de error con respecto al vector de parámetros θ , y va en la dirección del gradiente descendente. Una vez que el gradiente es cero, ¡ha alcanzado un mínimo!

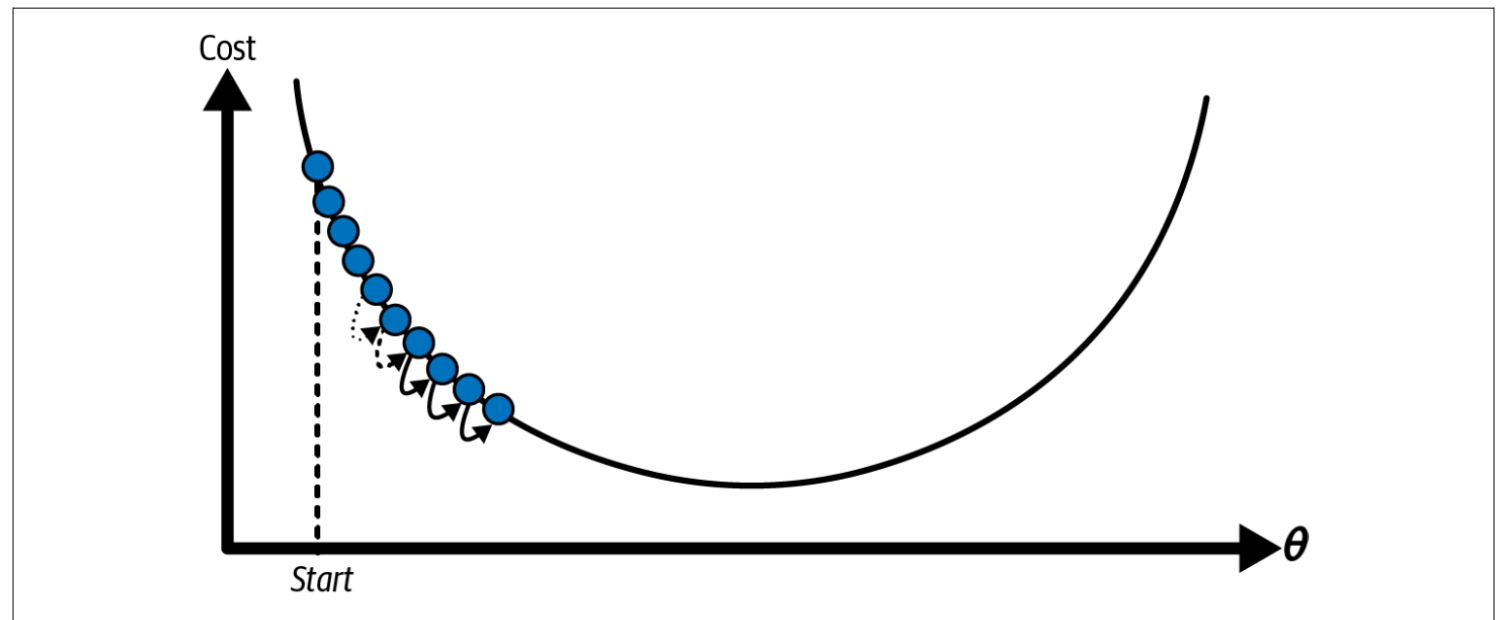
Descenso de gradiente

- En la práctica, se empieza llenando θ con valores aleatorios (esto se llama *inicialización aleatoria*).
- A continuación, se mejora gradualmente, dando un paso cada vez; cada paso intentando disminuir la función de coste (por ejemplo, el MSE), hasta que el algoritmo converge a un mínimo.



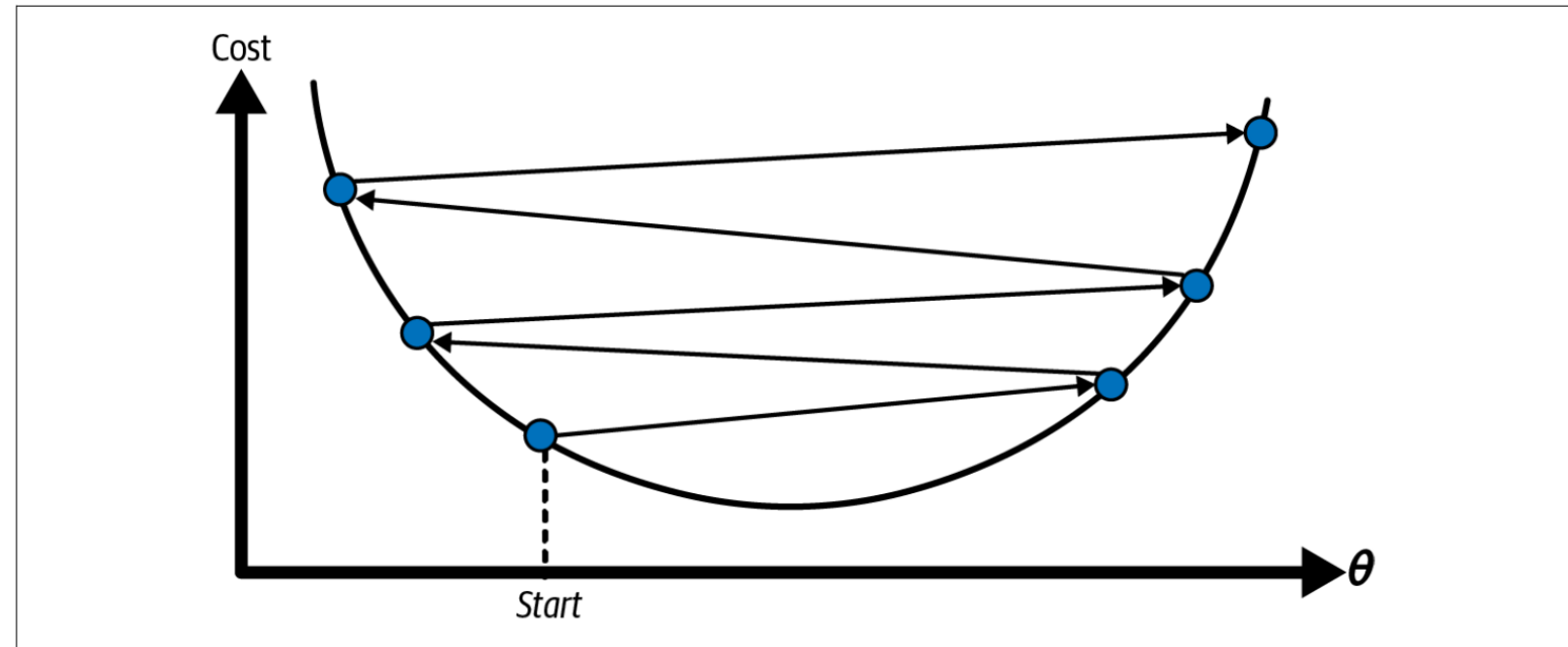
Descenso de gradiente

- Un parámetro importante en el descenso de gradiente es el tamaño de los pasos, determinado por el **hiperparámetro de tasa de aprendizaje (*learning rate*)**.
- Si la tasa de aprendizaje es demasiado pequeña, el algoritmo tendrá que pasar por muchas iteraciones para converger, lo que llevará mucho tiempo.



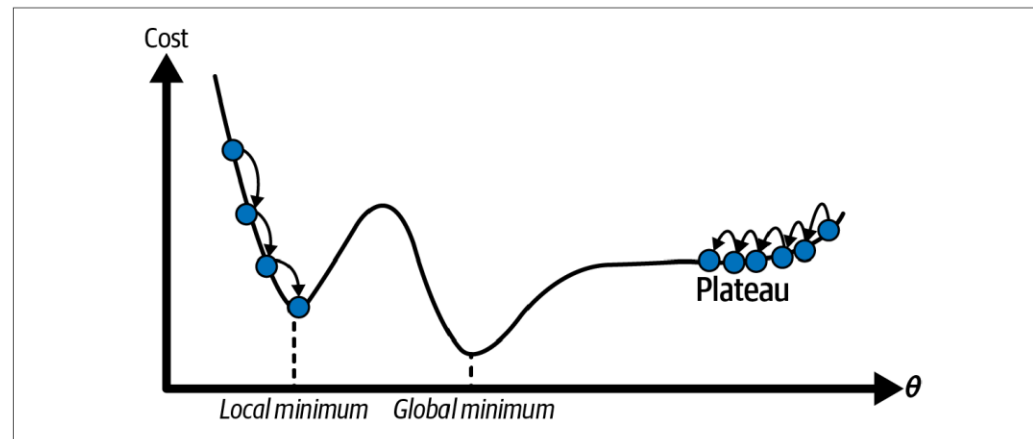
Descenso de gradiente

- Por otro lado, si el ritmo de aprendizaje es demasiado alto, puede que saltes al otro lado del valle y acabes en la otra orilla, posiblemente incluso más arriba de lo que estabas antes.
- Esto puede hacer que el algoritmo diverja, con valores cada vez mayores, sin encontrar una buena solución.



Descenso de gradiente

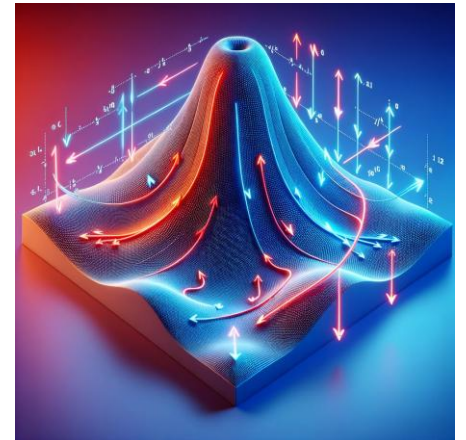
- Además, no todas las funciones de coste parecen cuencos bonitos y regulares. Puede haber agujeros, crestas, mesetas y todo tipo de terreno irregular, lo que dificulta la convergencia al mínimo.



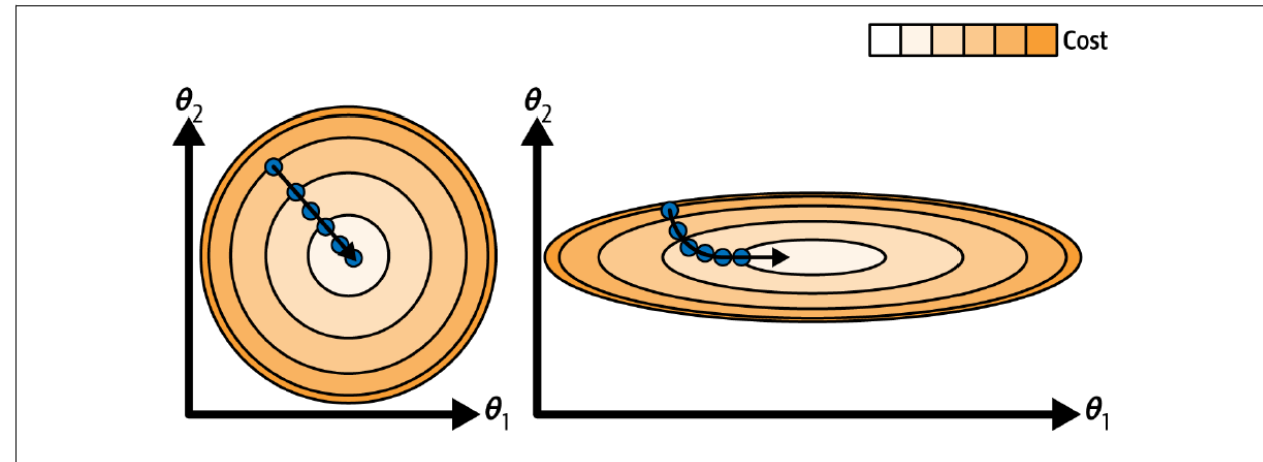
- Si la inicialización aleatoria inicia el algoritmo por la izquierda, entonces convergerá a un mínimo local, que no es tan bueno como el mínimo global. Si comienza por la derecha, entonces tardará mucho tiempo en cruzar la meseta. Y si se detiene demasiado pronto, nunca alcanzará el mínimo global.

Descenso de gradiente

- Afortunadamente, la función de coste MSE para un modelo de regresión lineal resulta ser una *función convexa*, lo que significa que si se eligen dos puntos cualesquiera de la curva, el segmento de línea que los une nunca está por debajo de la curva.
- Esto implica que no hay mínimos locales, sólo un mínimo global. También es una función continua con una pendiente que nunca cambia bruscamente.
- Por lo tanto: se garantiza que el descenso de gradiente se aproxime arbitrariamente al mínimo global.



Descenso de gradiente



- A la izquierda, el algoritmo de descenso por gradiente se dirige directamente hacia el mínimo, alcanzándolo así rápidamente, mientras que a la derecha se dirige primero en una dirección casi ortogonal a la del mínimo global, y termina con una larga marcha por un valle casi llano.
- Acabará alcanzando el mínimo, pero tardará mucho tiempo.
- Cuando utilices el descenso de gradiente, debe asegurarse de que todas las características tienen una escala similar (por ejemplo, utilizando la clase *StandardScaler* de Scikit-Learn), o de lo contrario tardará mucho más en converger.

Descenso de gradiente por lotes

- **Para aplicar el descenso de gradiente, es necesario calcular el gradiente de la función de coste con respecto a cada parámetro del modelo θ_j .**
- Tienes que calcular cuánto cambiará la función de coste si cambias θ_j sólo un poco. Esto se llama derivada parcial. Es como preguntar: "¿Cuál es la pendiente de la montaña bajo mis pies si miro hacia el este?" y luego hacer la misma pregunta mirando hacia el norte...
- Derivada parcial del MSE con respecto al parámetro θ_j :

$$\text{MSE}(\mathbf{X}, h_{\boldsymbol{\theta}}) = \frac{1}{m} \sum_{i=1}^m (\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)})^2$$

$$\frac{\partial}{\partial \theta_j} \text{MSE}(\boldsymbol{\theta}) = \frac{2}{m} \sum_{i=1}^m (\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)}) x_j^{(i)}$$

Descenso de gradiente por lotes

- En lugar de calcular estas derivadas parciales individualmente, puede utilizar la siguiente ecuación para calcularlas todas de una sola vez. El vector gradiente, anotado $\nabla_{\theta} \text{MSE}(\theta)$, contiene todas las derivadas parciales de la función de coste (una por cada parámetro del modelo).

$$\nabla_{\theta} \text{MSE}(\theta) = \begin{pmatrix} \frac{\partial}{\partial \theta_0} \text{MSE}(\theta) \\ \frac{\partial}{\partial \theta_1} \text{MSE}(\theta) \\ \vdots \\ \frac{\partial}{\partial \theta_n} \text{MSE}(\theta) \end{pmatrix} = \frac{2}{m} \mathbf{X}^T (\mathbf{X}\theta - \mathbf{y})$$

Descenso de gradiente por lotes

$$\nabla_{\boldsymbol{\theta}} \text{MSE}(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial}{\partial \theta_0} \text{MSE}(\boldsymbol{\theta}) \\ \frac{\partial}{\partial \theta_1} \text{MSE}(\boldsymbol{\theta}) \\ \vdots \\ \frac{\partial}{\partial \theta_n} \text{MSE}(\boldsymbol{\theta}) \end{pmatrix} = \frac{2}{m} \mathbf{X}^T (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$$

- Observe que esta fórmula implica cálculos sobre el conjunto completo de entrenamiento $\mathbf{X}$, ¡en cada paso del descenso de gradiente! Por eso el algoritmo se llama *descenso de gradiente por lotes*: utiliza el lote completo de datos de entrenamiento en cada paso.
- **Como resultado, es terriblemente lento en conjuntos de entrenamiento muy grandes.** Sin embargo, el descenso de gradiente escala bien con el número de características; **el entrenamiento de un modelo de regresión lineal cuando hay cientos de miles de características es mucho más rápido utilizando el descenso de gradiente que utilizando la ecuación Normal o la descomposición SVD.**

Descenso de gradiente por lotes

- Una vez que tengas el vector gradiente, que apunta cuesta arriba, sólo tienes que ir en la dirección opuesta para ir cuesta abajo.
- Esto significa restar $\nabla_{\theta} \text{MSE}(\theta)$ de θ .
- Aquí es donde entra en juego la tasa de aprendizaje η : multiplica el vector gradiente por η para determinar el tamaño del paso descendente:

$$\theta^{(\text{next step})} = \theta - \eta \nabla_{\theta} \text{MSE}(\theta)$$

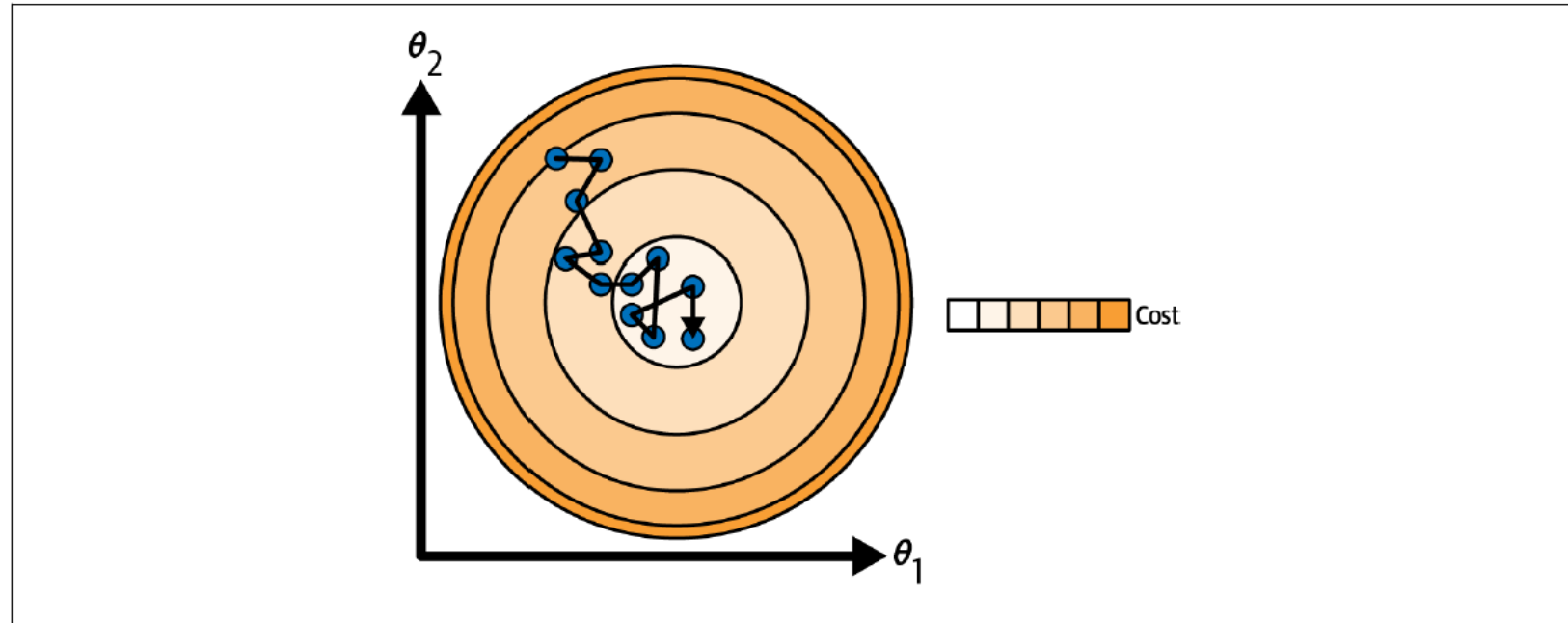
Descenso de gradiente por lotes

- Para encontrar una buena tasa de aprendizaje, puedes utilizar la búsqueda en cuadrícula (**grid search**). Sin embargo, es posible que desees limitar el número de épocas para que la búsqueda de cuadrícula pueda eliminar los modelos que tardan demasiado en converger.
- Puede que te preguntes cómo establecer el número de épocas.
- Si es demasiado bajo, seguirá estando lejos de la solución óptima cuando el algoritmo se detenga; pero si es demasiado alto, perderá tiempo mientras los parámetros del modelo ya no cambian.
- Una solución sencilla consiste en fijar un número muy elevado de épocas, pero interrumpir el algoritmo cuando el vector gradiente se vuelve diminuto - es decir, cuando su norma se hace más pequeña que un número minúsculo ϵ (llamado tolerancia)-, porque esto ocurre cuando el descenso gradiente ha alcanzado (casi) el mínimo.

Descenso de gradiente Estocástico

- El principal problema del descenso de gradiente por lotes es que utiliza todo el conjunto de entrenamiento para calcular los gradientes en cada paso, lo que lo hace muy lento cuando el conjunto de entrenamiento es grande.
- **El descenso de gradiente estocástico elige una instancia aleatoria en el conjunto de entrenamiento en cada paso y calcula los gradientes basándose únicamente en esa única instancia.**
- Trabajar con una sola instancia hace que el algoritmo sea mucho más rápido porque tiene muy pocos datos que manipular en cada iteración.
- También permite entrenar conjuntos de entrenamiento enormes, ya que sólo es necesario que haya una instancia en memoria en cada iteración.
- Por otra parte, debido a su naturaleza estocástica (es decir, aleatoria), este algoritmo es mucho menos regular que el descenso de gradiente por lotes: en lugar de disminuir suavemente hasta alcanzar el mínimo, la función de coste rebotará hacia arriba y hacia abajo, disminuyendo sólo en promedio.
- Una vez que el algoritmo se detiene, los valores finales de los parámetros serán buenos, pero no óptimos.

Descenso de gradiente Estocástico



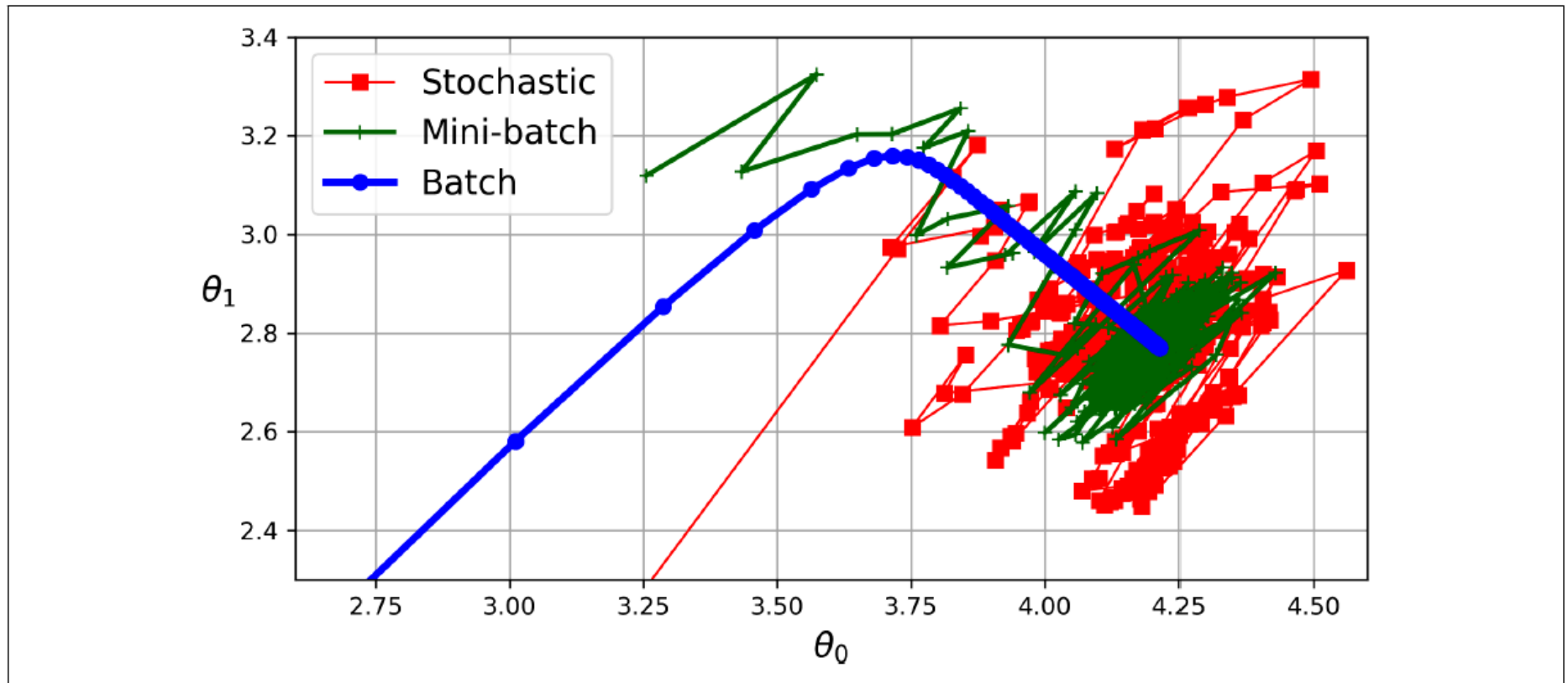
- La aleatoriedad es buena para escapar de los óptimos locales, pero mala porque significa que el algoritmo nunca puede establecerse en el mínimo.
- Una solución a este dilema es reducir gradualmente el ritmo de aprendizaje.
- Este proceso es similar al *enfriamiento simulado*.
- La función que determina el ritmo de aprendizaje en cada iteración se denomina *programa de aprendizaje (learning schedule)*.

Mini-lote de descenso de gradiente

- El descenso de gradiente por mini lotes es sencillo una vez que se conoce el descenso de gradiente por lotes y estocástico: en cada paso, el GD por mini **lotes calcula los gradientes en pequeños conjuntos aleatorios de instancias llamados mini lotes.**
- La principal ventaja de la GD minilotes sobre la GD estocástica es que se puede obtener un aumento del rendimiento gracias a la optimización por hardware de las operaciones matriciales, especialmente cuando se utilizan GPUs.
- El progreso del algoritmo en el espacio de parámetros es menos errático que con el GD estocástico, especialmente con minilotes bastante grandes.
- Como resultado, el GD de minilotes acabará caminando un poco más cerca del mínimo que el GD estocástico.

Mini-lote de descenso de gradiente

- La siguiente figura muestra los caminos tomados por los tres algoritmos de descenso de gradiente en el espacio de parámetros durante el entrenamiento.



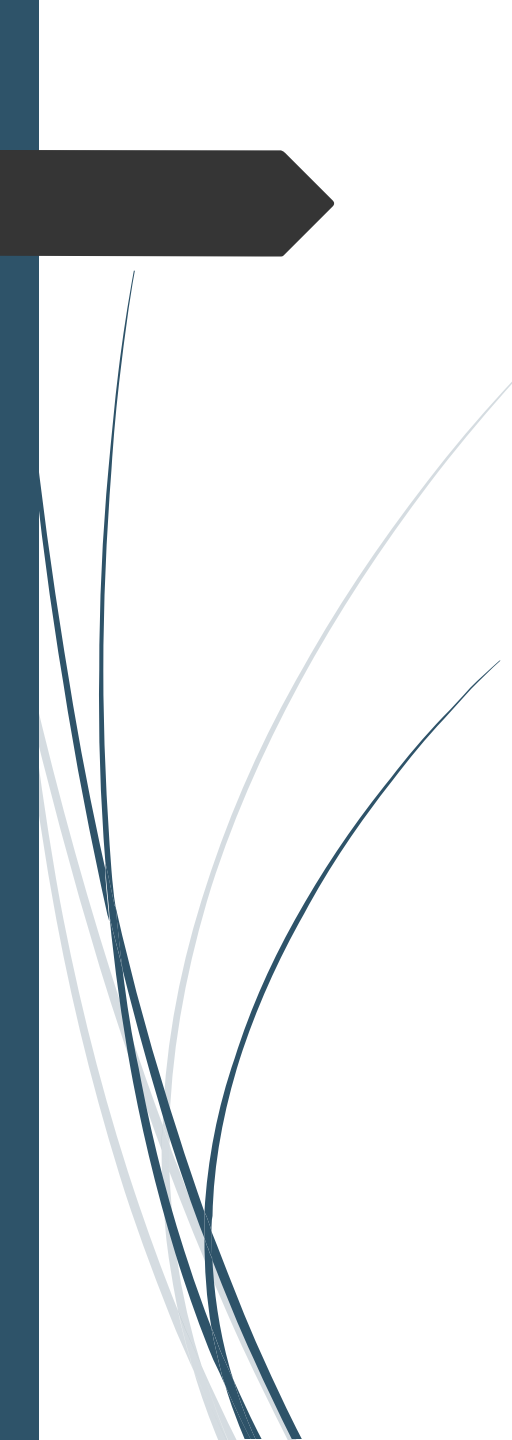
Mini-lote de descenso de gradiente

- Esta tabla compara los algoritmos que hemos discutido hasta ahora para la regresión lineal*.

Algorithm	Large m	Out-of-core support	Large n	Hyperparams	Scaling required	Scikit-Learn
Normal equation	Fast	No	Slow	0	No	N/A
SVD	Fast	No	Slow	0	No	LinearRegression
Batch GD	Slow	No	Fast	2	Yes	N/A
Stochastic GD	Fast	Yes	Fast	≥ 2	Yes	SGDRegressor
Mini-batch GD	Fast	Yes	Fast	≥ 2	Yes	N/A

- Casi no hay diferencias después del entrenamiento: todos estos algoritmos acaban con modelos muy similares y hacen predicciones exactamente de la misma manera.

* Mientras que la ecuación Normal sólo puede realizar regresión lineal, los algoritmos de descenso de gradiente pueden utilizarse para entrenar muchos otros modelos.

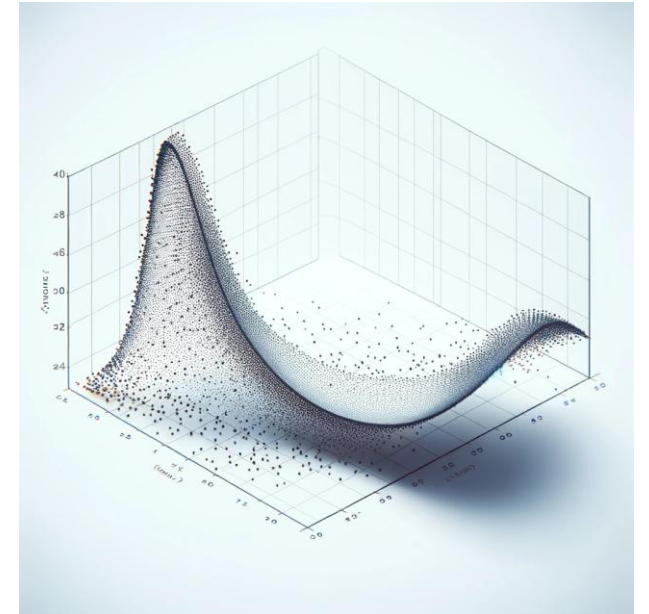


Aspectos avanzados con gradiente descendiente

- Regresión polinómica
- Curvas de aprendizaje
- El equilibrio entre sesgo y varianza
- Modelos lineales regularizados
- Regresión Ridge
- Regresión Lasso
- Regresión de red elástica (elastic-net)
- Parada anticipada (Early Stopping)
- Regresión logística
- Regresión Softmax

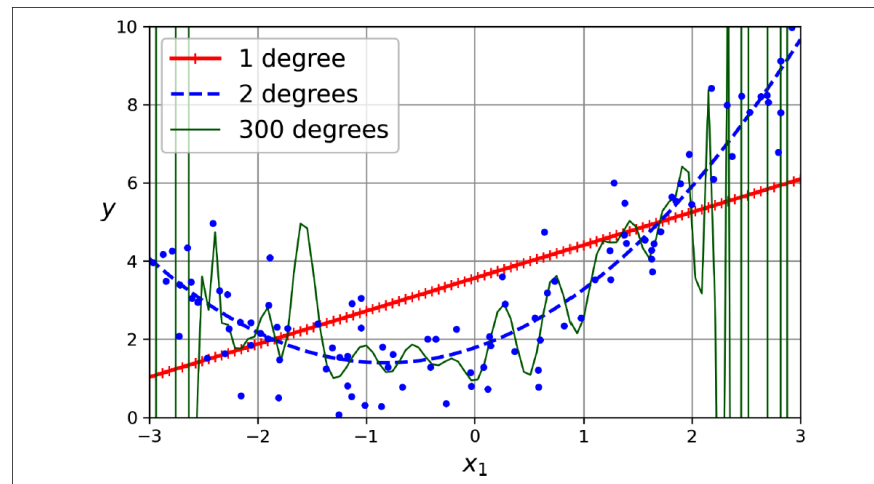
Regresión polinómica

- ¿Y si tus datos son más complejos que una línea recta?
- Sorprendentemente, se puede utilizar un modelo lineal para ajustar datos no lineales.
- Una forma sencilla de hacerlo es añadir potencias de cada característica como nuevas características y, a continuación, entrenar un modelo lineal en este conjunto ampliado de características.
- Esta técnica se denomina *regresión polinómica*.



Curvas de aprendizaje

- Si realizas una regresión polinómica de alto grado, es probable que se ajuste a los datos de entrenamiento mucho mejor que con una regresión lineal simple.
- En esta figura, aplicamos un modelo polinómico de 300 grados a los datos simulados que conforman los datos de entrenamiento de una ecuación de segundo grado, y comparamos el resultado con un modelo lineal puro y un modelo cuadrático.
- Observa cómo el modelo polinómico de 300 grados se contonea para acercarse lo más posible a las instancias de entrenamiento.



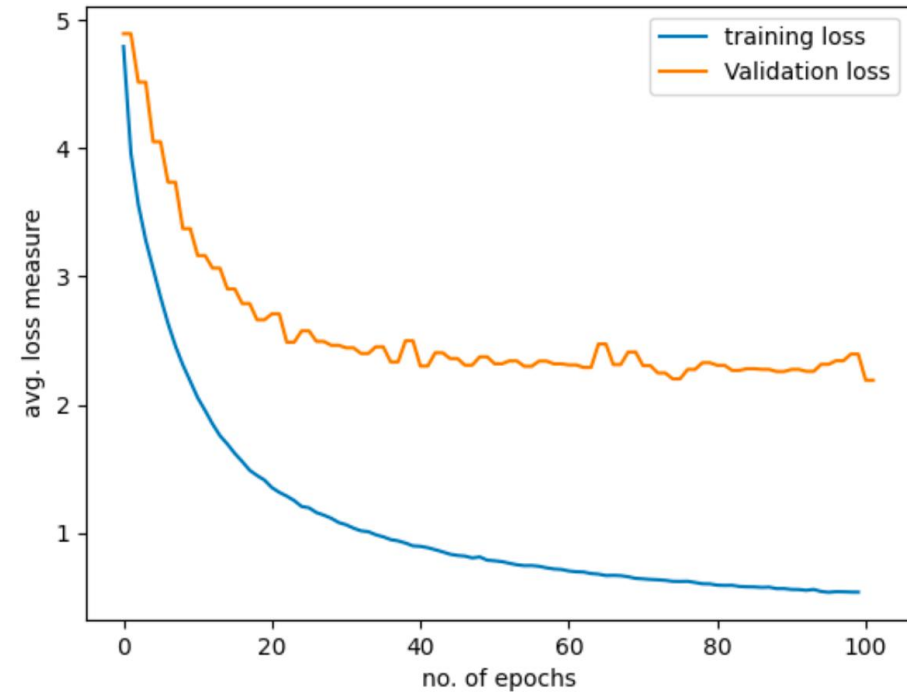


Curvas de aprendizaje

- Este modelo de regresión polinómica de alto grado se ajusta en exceso a los datos de entrenamiento, mientras que el modelo lineal se ajusta por debajo.
- El modelo que mejor generalizará en este caso es el modelo cuadrático, lo que es obvio porque los datos se generaron artificialmente utilizando un modelo cuadrático.
- Pero en general no sabrás qué función generó los datos, así que ¿cómo puedes decidir lo complejo que debe ser tu modelo? ¿Cómo saber si el modelo se ajusta demasiado o demasiado poco a los datos?
- Una forma es utilizar la validación cruzada para obtener una estimación del rendimiento de generalización de un modelo. Si un modelo funciona bien en los datos de entrenamiento pero generaliza mal según las métricas de validación cruzada, entonces el modelo está sobreajustado. Si funciona mal en ambos casos, el ajuste es insuficiente.

Curvas de aprendizaje

- ▶ Otra forma de saberlo es observar las *curvas de aprendizaje*, que son gráficos del error de entrenamiento y el error de validación del modelo en función de la iteración de entrenamiento:
- ▶ Sólo tienes que evaluar el modelo a intervalos regulares durante el entrenamiento, tanto en el conjunto de entrenamiento como en el conjunto de validación, y representar gráficamente los resultados. Si el modelo no puede entrenarse de forma incremental (es decir, si no admite *partial_fit()* o *warm_start*), deberá entrenarlo varias veces en subconjuntos gradualmente mayores del conjunto de entrenamiento.



El equilibrio entre sesgo y varianza

- Un importante resultado teórico de la estadística y el aprendizaje automático es el hecho de que el error de generalización de un modelo puede expresarse como la suma de tres errores muy diferentes:
- **Sesgo**
 - Esta parte del error de generalización se debe a suposiciones erróneas, como suponer que los datos son lineales cuando en realidad son cuadráticos. Lo más probable es que un modelo con un sesgo alto no se ajuste a los datos de entrenamiento.
- **Desviación**
 - Esta parte se debe a la excesiva sensibilidad del modelo a pequeñas variaciones en los datos de entrenamiento. Es probable que un modelo con muchos grados de libertad (como un modelo polinómico de alto grado) tenga una varianza elevada y, por tanto, sobreajuste los datos de entrenamiento.
- **Error irreducible**
 - **Esta parte se debe al ruido de los propios datos. La única forma de reducir esta parte del error es limpiar los datos.**
 - Aumentar la complejidad de un modelo suele aumentar su varianza y reducir su sesgo. A la inversa, reducir la complejidad de un modelo aumenta su sesgo y reduce su varianza. Por eso se denomina equilibrio.

Modelos lineales regularizados

- Una buena forma de **reducir el sobreajuste es regularizar el modelo (es decir, restringirlo)**: cuantos menos grados de libertad tenga, más difícil será que sobreajuste los datos.
- Una forma sencilla de regularizar un modelo polinómico es reducir el número de grados del polinomio.
- **Para un modelo lineal, la regularización se consigue normalmente restringiendo los pesos del modelo.**

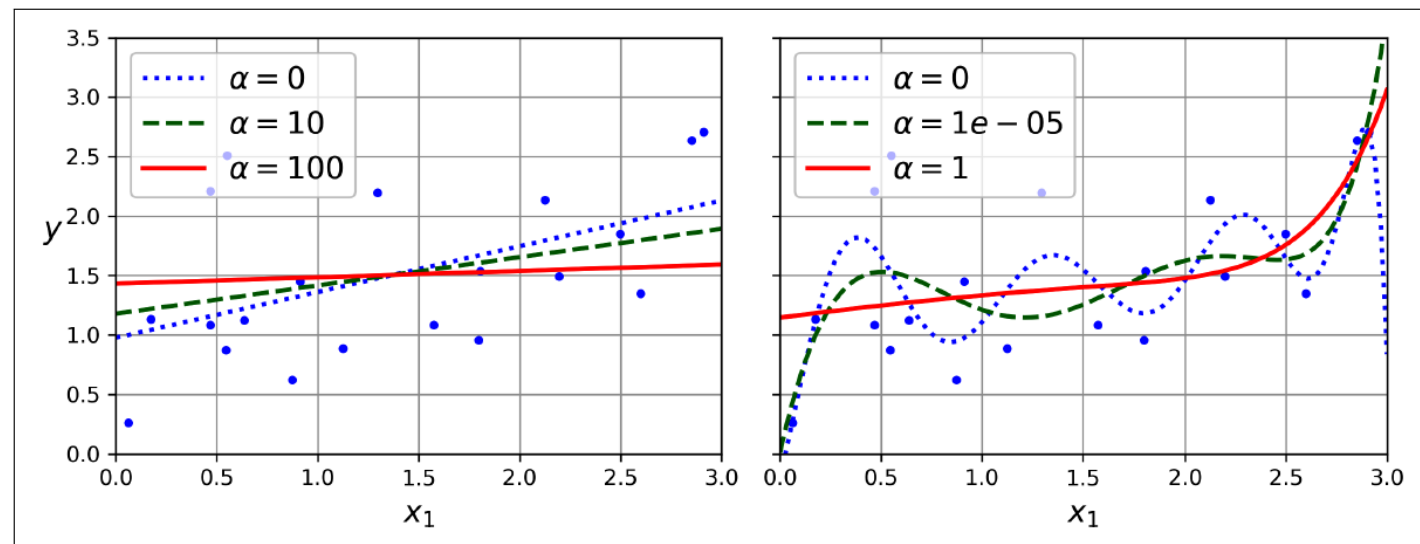
Regresión Ridge

- **La regresión Ridge es una versión regularizada de la regresión lineal: se añade un término de regularización al MSE. Esto obliga al algoritmo de aprendizaje no sólo a ajustarse a los datos, sino también a mantener las ponderaciones del modelo lo más pequeñas posible.**
- Ten en cuenta que el término de regularización sólo debe añadirse a la función de coste durante el entrenamiento. Una vez entrenado el modelo, puede utilizar el MSE no regularizado (o el RMSE) para evaluar el rendimiento del modelo.
- El hiperparámetro α controla el grado de regularización del modelo. Si $\alpha = 0$, la regresión ridge es simplemente una regresión lineal. Si α es muy grande, todas las ponderaciones terminan muy cerca de cero y el resultado es una línea plana que pasa por la media de los datos.

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + \frac{\alpha}{m} \sum_{i=1}^n \theta_i^2$$

Regresión Ridge

- Obsérvese que el término de sesgo θ_0 no está regularizado (la suma comienza en $i = 1$, no en 0).
- Si definimos $\mathbf{w}$ como el vector de pesos de características (θ_1 a θ_n), entonces el término de regularización es igual a $\alpha(\|\mathbf{w}\|_2)^2 / m$, donde $\|\mathbf{w}\|_2$ representa la norma ℓ_2 del vector de pesos.
- Para el descenso de gradiente por lotes, basta con añadir $2\alpha\mathbf{w} / m$ a la parte del vector de gradiente MSE que corresponde a los pesos de las características, sin añadir nada al gradiente del término de sesgo.



Regresión Ridge

- Al igual que con la regresión lineal, podemos realizar la regresión Ridge calculando una ecuación de forma cerrada o realizando el descenso de gradiente.
- Los pros y los contras son los mismos.
- La siguiente ecuación muestra la solución de forma cerrada, donde $\mathbf{A}$ es la matriz de identidad $(n + 1) \times (n + 1)$, excepto con un 0 en la celda superior izquierda, que corresponde al término de sesgo.

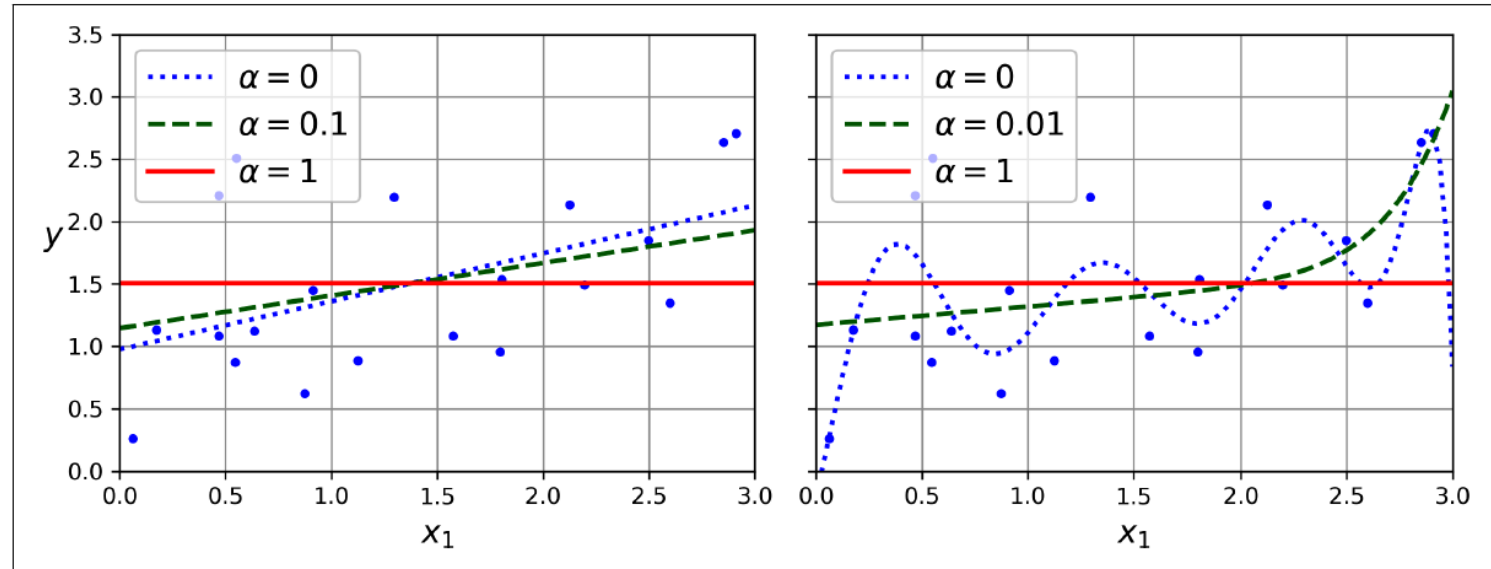
$$\hat{\boldsymbol{\theta}} = (\mathbf{X}^T \mathbf{X} + \alpha \mathbf{A})^{-1} \mathbf{X}^T \mathbf{y}$$

Regresión Lasso

- La regresión del *mínimo absoluto* y del operador de selección (normalmente llamada simplemente *regresión lasso*) es otra versión regularizada de la regresión lineal.
- Al igual que la regresión ridge, añade un término de regularización a la función de costes, pero utiliza la norma ℓ_1 del vector de pesos en lugar del cuadrado de la norma ℓ_2 .
- Observe que la norma ℓ_1 se multiplica por 2α , mientras que la norma ℓ_2 se multiplicó por α / m en la regresión ridge. Estos factores se eligieron para garantizar que el valor óptimo de α es independiente del tamaño del conjunto de entrenamiento: diferentes normas conducen a diferentes factores.

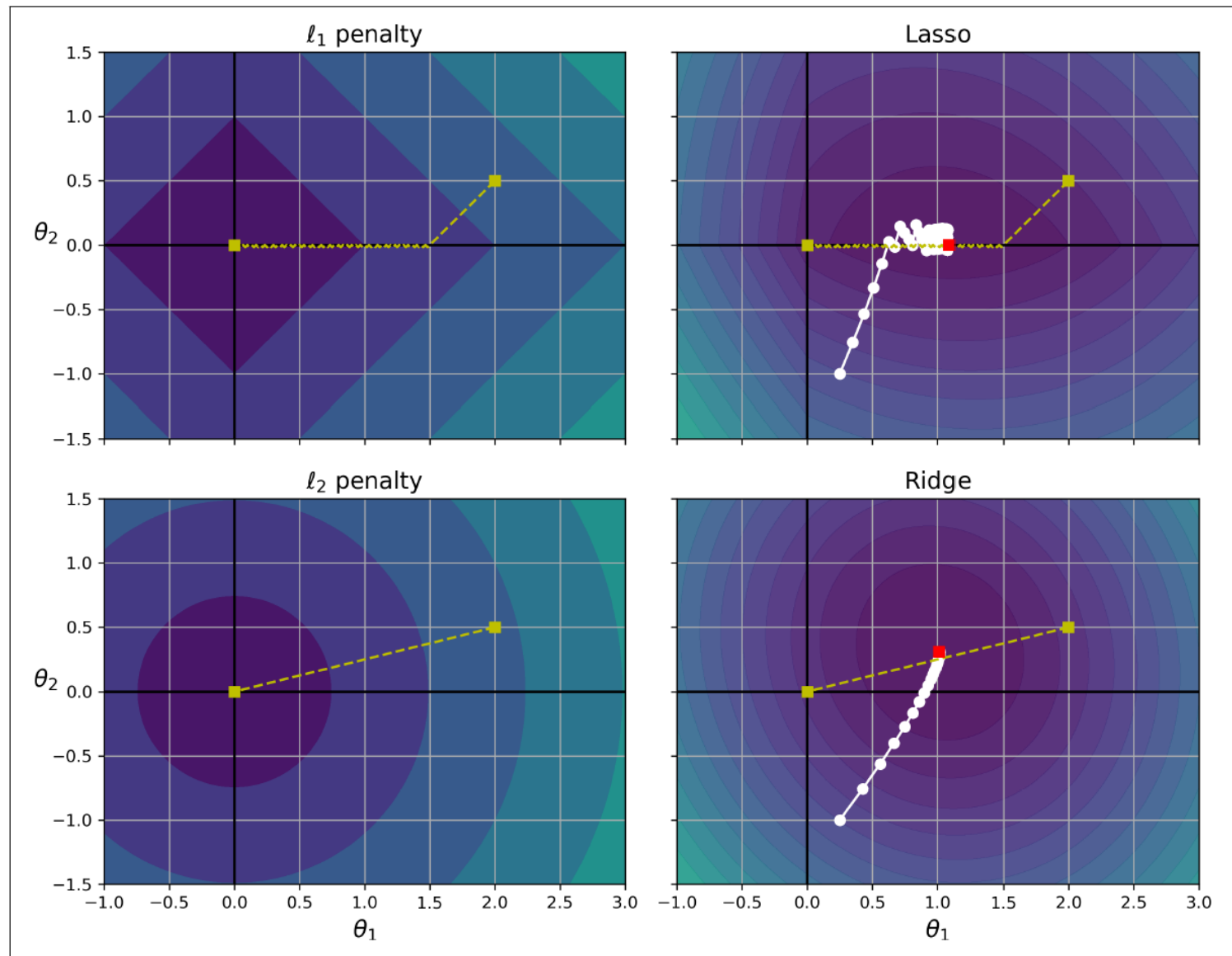
$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + 2\alpha \sum_{i=1}^n |\theta_i|$$

Regresión Lasso



- Una característica importante de la regresión lasso es que tiende a eliminar las ponderaciones de las características menos importantes (es decir, las pone a cero).
- La línea discontinua del gráfico de la derecha (con $\alpha = 0,01$) parece aproximadamente cúbica: todas las ponderaciones de las características polinómicas de alto grado son iguales a cero.
- En otras palabras, la regresión lasso realiza automáticamente la selección de características y produce un modelo disperso con pocas ponderaciones de características distintas de cero.

Regresión Lasso



Regresión Lasso

- La función de coste de Lasso no es diferenciable en $\theta_i = 0$ (para $i = 1, 2, \dots, n$), pero el descenso gradiente sigue funcionando si en su lugar se utiliza un *vector subgradiente* **g** cuando cualquier $\theta_i = 0$.
- La siguiente ecuación muestra una ecuación de vector subgradiente que se puede utilizar para el descenso gradiente con la función de coste lazo.

$$g(\boldsymbol{\theta}, J) = \nabla_{\boldsymbol{\theta}} \text{MSE}(\boldsymbol{\theta}) + 2\alpha \begin{pmatrix} \text{sign}(\theta_1) \\ \text{sign}(\theta_2) \\ \vdots \\ \text{sign}(\theta_n) \end{pmatrix} \quad \text{where } \text{sign}(\theta_i) = \begin{cases} -1 & \text{if } \theta_i < 0 \\ 0 & \text{if } \theta_i = 0 \\ +1 & \text{if } \theta_i > 0 \end{cases}$$

Regresión de red elástica (elastic-net)

- **La regresión de red elástica es un término medio entre la regresión ridge y la regresión lasso.**
- El término de regularización es una suma ponderada de los términos de regularización de ridge y lasso, y se puede controlar la relación de mezcla con el hiperparámetro r .
- Cuando $r = 0$, la red elástica equivale a la regresión ridge, y cuando $r = 1$, equivale a la regresión lasso

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + r(2\alpha \sum_{i=1}^n |\theta_i|) + (1 - r)\left(\frac{\alpha}{m} \sum_{i=1}^n \theta_i^2\right)$$

Regresión de red elástica

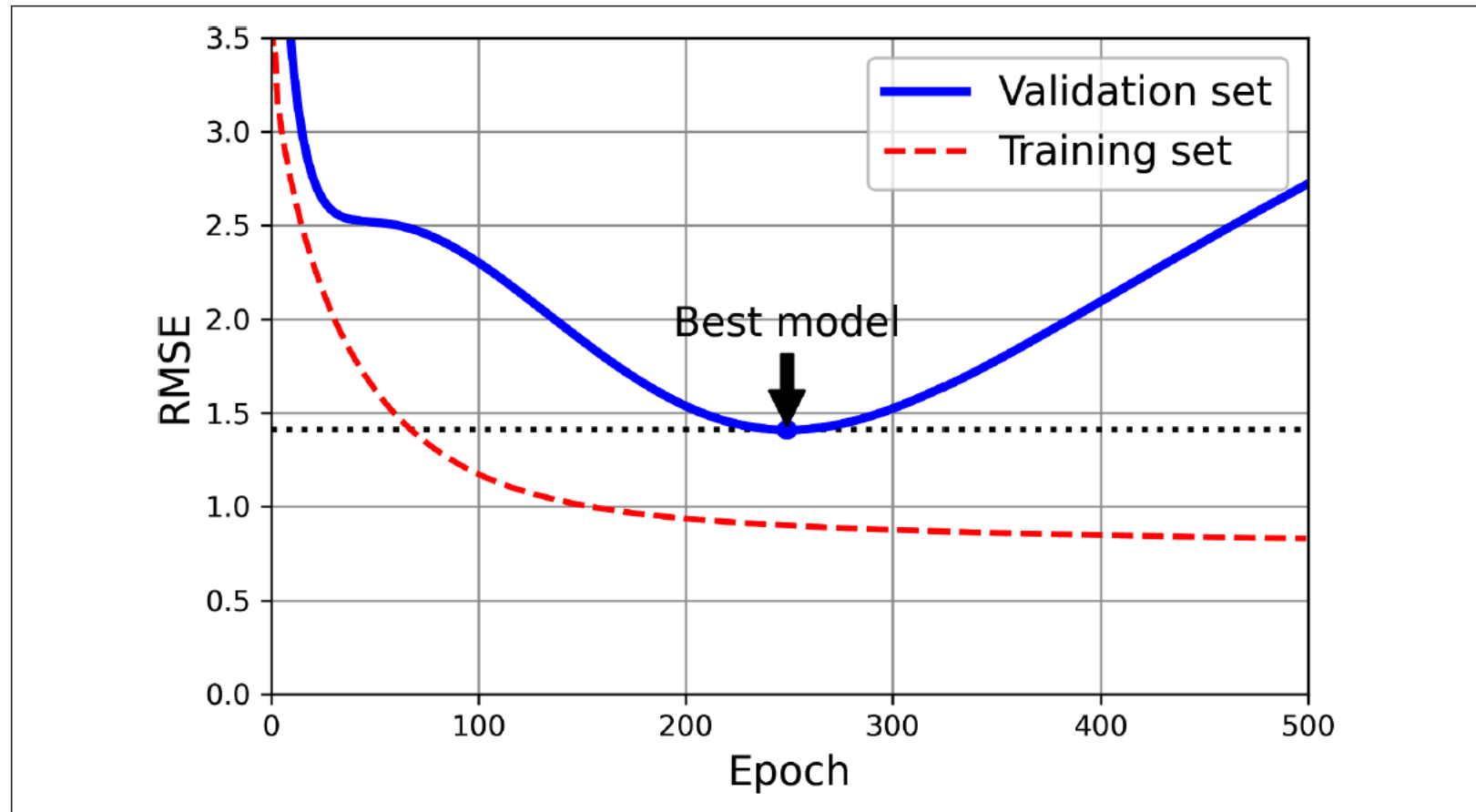
- ¿Cuándo se debe utilizar la regresión de red elástica, la regresión ridge, lasso o la regresión lineal simple (es decir, sin regularización)?
- Casi siempre es preferible tener al menos un poco de regularización, por lo que generalmente se debe evitar la regresión lineal simple.
- Ridge es una buena opción por defecto, pero si sospecha que sólo unas pocas características son útiles, debería preferir lasso o elastic net porque tienden a reducir los pesos de las características inútiles a cero, como se ha comentado anteriormente.
- En general, se prefiere la red elástica al lasso porque éste puede comportarse de forma errática cuando el número de características es mayor que el número de instancias de entrenamiento o cuando varias características están fuertemente correlacionadas.

Parada anticipada (Early Stopping)

- Una forma muy distinta de **regularizar algoritmos de aprendizaje iterativo como el descenso de gradiente es detener el entrenamiento en cuanto el error de validación alcance un mínimo**. Es lo que se denomina detención temprana.
- La figura de la siguiente diapositiva muestra un modelo complejo (en este caso, un modelo de regresión polinómica de alto grado) entrenado con descenso de gradiente por lotes en un conjunto de dato sintético con funciones cuadráticas.
- A medida que pasan las épocas, el algoritmo aprende y su error de predicción (RMSE) en el conjunto de entrenamiento disminuye, junto con su error de predicción en el conjunto de validación.
- Sin embargo, al cabo de un tiempo, el error de validación deja de disminuir y empieza a aumentar. Esto indica que el modelo ha empezado a sobreajustar los datos de entrenamiento.
- Con la parada anticipada se detiene el entrenamiento en cuanto el error de validación alcanza el mínimo.

Parada anticipada

► Regularización de parada anticipada



Regresión logística

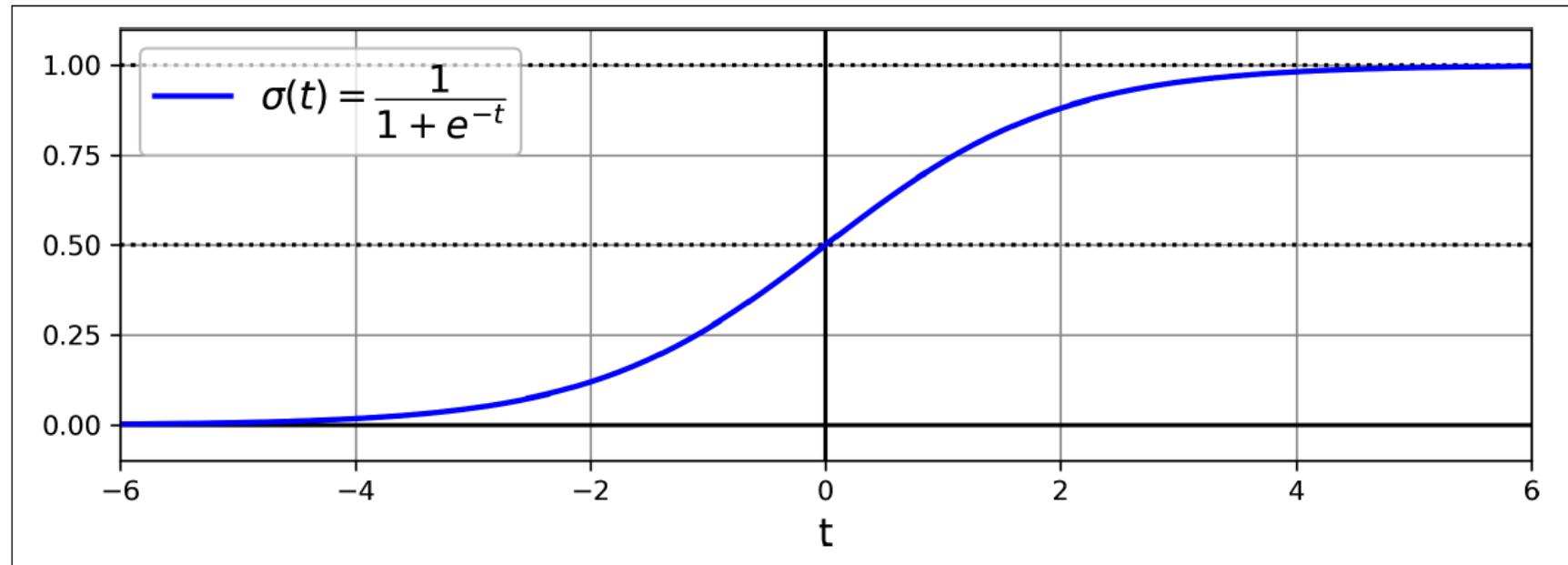
- Algunos algoritmos de regresión pueden utilizarse para la clasificación (y viceversa). **La regresión logística se utiliza habitualmente para estimar la probabilidad de que una instancia pertenezca a una clase determinada.**
- Si la probabilidad estimada es superior a un umbral determinado (normalmente, el 50%), el modelo predice que la instancia pertenece a esa clase (denominada clase positiva, etiquetada como "1") y, en caso contrario, predice que no (es decir, que pertenece a la clase negativa, etiquetada como "0"). Se trata, por tanto, de un clasificador binario.
- Un modelo de regresión logística calcula una suma ponderada de las características de entrada (más un término de sesgo), pero en lugar de emitir el resultado directamente como hace el modelo de regresión lineal, emite la *logística* de este resultado.

$$\hat{p} = h_{\theta}(\mathbf{x}) = \sigma(\boldsymbol{\theta}^T \mathbf{x})$$

Regresión logística

- La función logística -notada $\sigma(\cdot)$ - es una función sigmoidea (es decir, con forma de S) que da como resultado un número entre 0 y 1.

$$\sigma(t) = \frac{1}{1 + \exp(-t)}$$



Regresión logística

- Una vez que el modelo de regresión logística ha estimado la probabilidad $p = h_{\theta}(\mathbf{x})$ de que una instancia $\mathbf{x}$ pertenezca a la clase positiva, puede realizar su predicción $\hat{y}$ fácilmente:

$$\hat{y} = \begin{cases} 0 & \text{if } \hat{p} < 0.5 \\ 1 & \text{if } \hat{p} \geq 0.5 \end{cases}$$

- Observa que $\sigma(t) < 0.5$ cuando $t < 0$, y $\sigma(t) \geq 0.5$ cuando $t \geq 0$, por lo que un modelo de regresión logística que utilice el umbral predeterminado del 50% de probabilidad predice 1 si $\theta_{\top} \mathbf{x}$ es positivo y 0 si es negativo.

Regresión logística: Función de entrenamiento y coste

- El objetivo del entrenamiento es establecer el vector de parámetros θ de modo que el modelo estime probabilidades altas para los casos positivos ($y = 1$) y probabilidades bajas para los casos negativos ($y = 0$).
- Esta idea se recoge en la función de coste que se muestra en la siguiente ecuación para una única instancia de entrenamiento $\mathbf{x}$.

$$c(\theta) = \begin{cases} -\log(\hat{p}) & \text{if } y = 1 \\ -\log(1 - \hat{p}) & \text{if } y = 0 \end{cases}$$

- Esta función de coste tiene sentido porque $-\log(t)$ crece mucho cuando t se acerca a 0, por lo que el coste será grande si el modelo estima una probabilidad cercana a 0 para un caso positivo, y también será grande si el modelo estima una probabilidad cercana a 1 para un caso negativo. Por otro lado, $-\log(t)$ se aproxima a 0 cuando t se aproxima a 1, por lo que el coste se aproximará a 0 si la probabilidad estimada se aproxima a 0 para un caso negativo o se aproxima a 1 para un caso positivo, que es precisamente lo que queremos.

Regresión logística: Función de entrenamiento y coste

- La función de coste sobre todo el conjunto de entrenamiento es el coste medio sobre todas las instancias de entrenamiento. Se puede escribir en una única expresión denominada *pérdida logarítmica*.

$$J(\boldsymbol{\theta}) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{p}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)}) \right]$$

- Se puede demostrar matemáticamente que la minimización de esta pérdida dará como resultado el modelo con la máxima probabilidad de ser óptimo, suponiendo que las instancias siguen una distribución gaussiana alrededor de la media de su clase. Cuando se utiliza la pérdida logarítmica, ésta es la suposición implícita que se hace.
- Cuanto más errónea sea esta suposición, más sesgado será el modelo. Del mismo modo, cuando utilizamos el MSE para entrenar modelos de regresión lineal, estábamos suponiendo implícitamente que los datos eran puramente lineales, más algo de ruido gaussiano.

Regresión logística: Función de entrenamiento y coste

- No se conoce ninguna ecuación de forma cerrada para calcular el valor de θ que minimiza esta función de coste (no existe un equivalente de la ecuación Normal).
- Pero esta función de coste es convexa, por lo que el descenso gradiente (o cualquier otro algoritmo de optimización) tiene garantizado encontrar el mínimo global.
- Las derivadas parciales de la función de coste con respecto al parámetro del modelo j^{th} θ_j vienen dadas por:

$$\frac{\partial}{\partial \theta_j} J(\theta) = \frac{1}{m} \sum_{i=1}^m \left(\sigma(\theta^T \mathbf{x}^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

- Ecuación similar al descenso por gradiente visto anteriormente.

Regresión Softmax

- El modelo de regresión logística puede generalizarse para admitir varias clases directamente, sin tener que entrenar y combinar varios clasificadores binarios.
- Esto se denomina *regresión softmax* o *regresión logística multinomial*.
- La idea es sencilla: dada una instancia $\mathbf{x}$, el modelo de regresión softmax calcula primero una puntuación $s_k(\mathbf{x})$ para cada clase k y, a continuación, estima la probabilidad de cada clase aplicando la *función softmax* (también denominada *exponencial normalizada*) a las puntuaciones. La ecuación para calcular $s_k(\mathbf{x})$ debería resultarte familiar, ya que es igual que la ecuación para la predicción de regresión lineal.

$$s_k(\mathbf{x}) = \left(\boldsymbol{\theta}^{(k)} \right)^T \mathbf{x}$$

- Ten en cuenta que cada clase tiene su propio vector de parámetros $\boldsymbol{\theta}^{(k)}$. Todos estos vectores se almacenan normalmente como filas de una *matriz de parámetros* Θ .

Regresión Softmax

- Una vez calculada la puntuación de cada clase para la instancia $\mathbf{x}$, puede estimar la probabilidad p_k de que la instancia pertenezca a la clase k ejecutando las puntuaciones a través de la función softmax.

$$\hat{p}_k = \sigma(\mathbf{s}(\mathbf{x}))_k = \frac{\exp(s_k(\mathbf{x}))}{\sum_{j=1}^K \exp(s_j(\mathbf{x}))}$$

- La función calcula el exponencial de cada puntuación y luego las normaliza (dividiendo por la suma de todos los exponenciales). Las puntuaciones suelen denominarse logits o log-odds.
- En esta ecuación:
 - K es el número de clases.
 - $\mathbf{s}(\mathbf{x})$ es un vector que contiene las puntuaciones de cada clase para la instancia $\mathbf{x}$.
 - $\sigma(\mathbf{s}(\mathbf{x}))_k$ es la probabilidad estimada de que la instancia $\mathbf{x}$ pertenezca a la clase k , dadas las puntuaciones de cada clase para esa instancia.

Regresión Softmax

- Al igual que el clasificador de regresión logística, por defecto el clasificador de regresión softmax predice la clase con la probabilidad estimada más alta.

$$\hat{y} = \operatorname{argmax}_k \sigma(\mathbf{s}(\mathbf{x}))_k = \operatorname{argmax}_k s_k(\mathbf{x}) = \operatorname{argmax}_k \left(\left(\boldsymbol{\theta}^{(k)} \right)^\top \mathbf{x} \right)$$

- En el entrenamiento, el objetivo es tener un modelo que estime una probabilidad alta para la clase objetivo (y, en consecuencia, una probabilidad baja para las demás clases).
- Minimizar la función de coste mostrada aquí, llamada *entropía cruzada*, debería conducir a este objetivo porque penaliza al modelo cuando estima una probabilidad baja para una clase objetivo.
- La entropía cruzada se utiliza con frecuencia para medir hasta qué punto un conjunto de probabilidades de clase estimadas coincide con las clases objetivo.

$$J(\boldsymbol{\Theta}) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log(\hat{p}_k^{(i)})$$

Regresión Softmax

- En esta ecuación, $y_k^{(i)}$ es la probabilidad objetivo de que la instancia i^{th} pertenezca a la clase k . En general, es igual a 1 o a 0, dependiendo de si la instancia pertenece a la clase o no.
- Observa que cuando sólo hay dos clases ($K = 2$), esta función de coste es equivalente a la función de coste de regresión logística.
- La entropía cruzada entre dos distribuciones de probabilidad p y q se define como $H(p, q) = -\sum_x p(x) \log q(x)$ (al menos cuando las distribuciones son discretas).
- El vector gradiente de esta función de coste con respecto a $\Theta^{(k)}$ viene dado por:

$$\nabla_{\Theta^{(k)}} J(\Theta) = \frac{1}{m} \sum_{i=1}^m \left(\hat{p}_k^{(i)} - y_k^{(i)} \right) \mathbf{x}^{(i)}$$

- Ahora puede calcular el vector de gradiente para cada clase y, a continuación, utilizar el descenso de gradiente (o cualquier otro algoritmo de optimización) para encontrar la matriz de parámetros Θ que minimiza la función de coste.



Aprendizaje supervisado

Introducción

Evaluación de modelos

Aprendizaje basado en instancias

Regresión Lineal

Modelo Naïve Bayes

Modelos basados en árboles de decisión

Modelos multclasificadores y de
descomposición

Aprendizaje basado en instancias:

Naïve Bayes

- El teorema de Bayes orientado a un problema de clasificación con n variables tiene la siguiente expresión.

$$P(C|A_1, \dots, A_n) = \frac{P(A_1, \dots, A_n|C)P(C)}{P(A_1, \dots, A_n)}$$

- Hipótesis MAP (*maximum a posteriori*): Si queremos clasificar un nuevo caso $(a_1, \dots, a_n)$ y la variable de clase C tiene k posibles categorías $\Omega = \{c_{C1}, \dots, c_k\}$, lo que nos interesa es identificar la más probable y devolverla como clasificación.

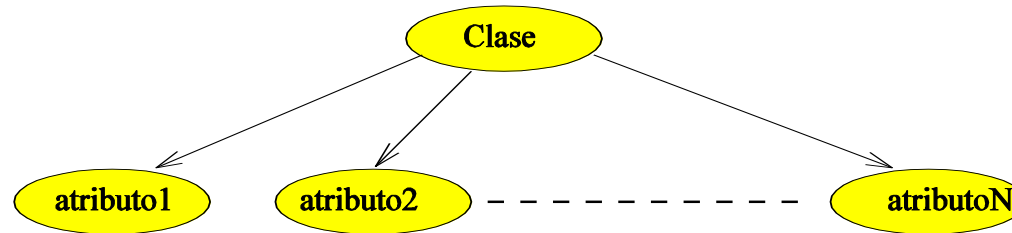
$$\begin{aligned} c_{MAP} &= \arg \max_{c \in \Omega_C} P(c|a_1, \dots, a_n) \\ &= \arg \max_{c \in \Omega_C} \frac{P(a_1, \dots, a_n|c)P(c)}{P(a_1, \dots, a_n)} \\ &= \arg \max_{c \in \Omega_C} P(a_1, \dots, a_n|c)P(c) \end{aligned}$$

- Problema: Tenemos que trabajar con la distribución conjunta y esto suele ser inmanejable.

Aprendizaje basado en instancias:

Naïve Bayes

- Naïve Bayes es el modelo de red bayesiana orientado a la clasificación más sencillo.
- Supone que todos los atributos son independientes conocidos de la variable de clase. Gráficamente



- En Naive Bayes (NB), el supuesto MAP es el siguiente:

$$c_{MAP} = \arg \max_{c \in \Omega_C} P(c|a_1, \dots, a_n) = \arg \max_{c \in \Omega_C} P(c) \prod_{i=1}^n P(a_i|c)$$

- A pesar de la suposición poco realista del NB, este algoritmo se considera un estándar y sus resultados son competitivos con la mayoría de los clasificadores.

Aprendizaje basado en instancias:

Naïve Bayes

- ¿Cómo se estiman estas probabilidades? Estimación de parámetros
- Variables discretas
- $P(x | c_i)$ se estima como la frecuencia relativa de ejemplos que tienen un valor dado de x_i y pertenecen a la clase c_i .
- Estimación de máxima verosimilitud (MLE). Sea $n(x_i)$ el número de veces que $X = x_{ij}$ aparece en la base de datos y $n(x_i, x_j)$ el número de veces que el par $(X_i = x_i, X_j = x_j)$ aparece en la base de datos, entonces

$$p(x_i | x_j) = \frac{n(x_i, x_j)}{n(x_j)}$$

- Suavizado mediante la corrección de Laplace:

$$p(x_i | x_j) = \frac{n(x_i, x_j) + 1}{n(x_j) + |\Omega_{x_i}|}$$

- Cuando hay muchos casos, tiende al MSE, si hay pocos casos tiende al uniforme.

Aprendizaje basado en instancias:

Naïve Bayes

Por ejemplo:

Día	Outlook	Temperatura	Humedad	Viento	¿Jugar al tenis?
1	Sunny	85	85	Luz	No
2	Sunny	80	90	Fuerte	No
3	Nublado	83	86	Luz	Sí
4	Lluvia	70	96	Luz	Sí
5	Lluvia	68	80	Luz	Sí
6	Lluvia	65	70	Fuerte	No
7	Nublado	64	65	Fuerte	Sí
8	Sunny	72	95	Luz	No
9	Sunny	69	70	Luz	Sí
10	Lluvia	75	80	Luz	Sí
11	Sunny	75	70	Fuerte	Sí
12	Nublado	72	90	Fuerte	Sí
13	Nublado	81	75	Luz	Sí
14	Lluvia	71	91	Fuerte	No

- Estimación para *PLAY*, outlook y windy

JUGAR	EMV	Lapl.	outlook	EMV		Lapl.		ventos o	EMV		Lapl.	
				no	sí	no	Sí		no	sí	no	Sí
no	5/14	6/16	soleado	3/5	2/9	4/8	3/12	verda dero	3/5	3/9	4/7	4/11
sí	9/14	10/16	nublado	0	4/9	1/8	5/12	falso	2/5	6/9	3/7	7/11
			lluvioso	2/5	3/9	3/8	4/12					

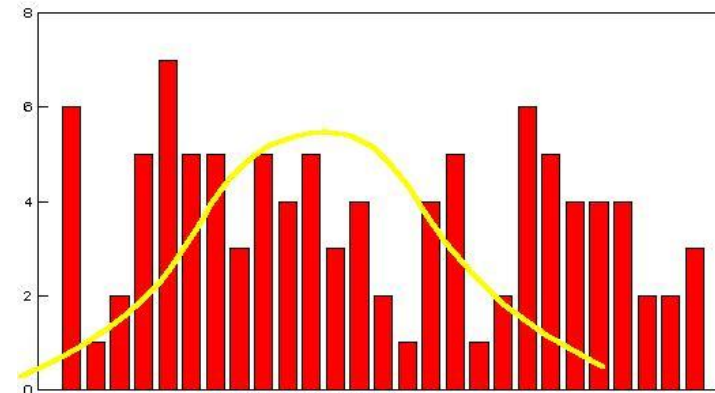
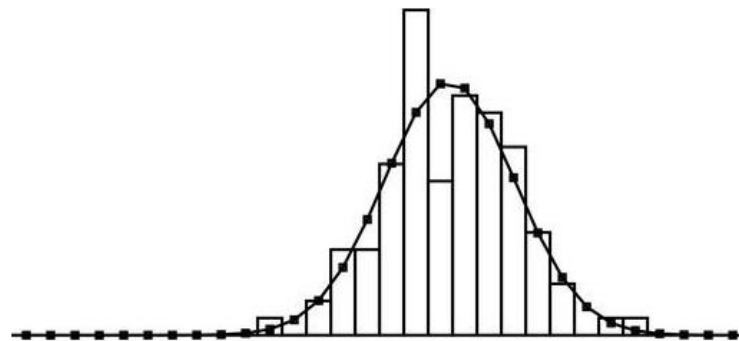
Aprendizaje basado en instancias:

Naïve Bayes

- Variables numéricas
- $P(x | c_i)$ se estima utilizando una función de densidad gaussiana. Es decir, se supone que los valores numéricos siguen una distribución normal

$$P(x|c_i) = N(x; \mu, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

- es decir, para cada categoría de la variable de clase se estima una distribución normal (de media μ y desviación típica σ).
- Evidentemente, en algunos casos la aproximación realizada será mejor que en otros.



Aprendizaje basado en instancias:

Naïve Bayes

Ejemplo:

Día	Outlook	Temperatura	Humedad	Viento	¿Jugar al tenis?
1	Sunny	85	85	Luz	No
2	Sunny	80	90	Fuerte	No
3	Nublado	83	86	Luz	Sí
4	Lluvia	70	96	Luz	Sí
5	Lluvia	68	80	Luz	Sí
6	Lluvia	65	70	Fuerte	No
7	Nublado	64	65	Fuerte	Sí
8	Sunny	72	95	Luz	No
9	Sunny	69	70	Luz	Sí
10	Lluvia	75	80	Luz	Sí
11	Sunny	75	70	Fuerte	Sí
12	Nublado	72	90	Fuerte	Sí
13	Nublado	81	75	Luz	Sí
14	Lluvia	71	91	Fuerte	No

► Estimación de la *temperatura* y la *humedad*

- temperatura=(PLAY=no): $\mu=74,6$; $\sigma=7,89$
- temperatura=(PLAY=sí): $\mu=73$; $\sigma=6,16$
- humedad=(PLAY=no): $\mu=86,6$; $\sigma=9,73$
- humedad=(PLAY=sí): $\mu=79,11$; $\sigma=10,21$



Aprendizaje basado en instancias:

Naïve Bayes

- Supongamos que queremos clasificar un nuevo caso:
 $x=(\text{panorama}=\text{soleado}, \text{temp}=87, \text{hum}=90, \text{ventoso}=\text{false})$
- Asumimos la estimación de máxima verosimilitud. Aplicamos para ambas categorías de la variable de clase:

- $\text{PLAY}=\text{no}$

$$\begin{aligned} P(x|\text{NO}) &= P(\text{NO}) \cdot P_{\text{O}}(\text{soleado}|\text{NO}) \cdot P_{\text{w}}(\text{falso}|\text{NO}) \cdot P_{\text{t}}(87|\text{NO}) \cdot P_{\text{h}}(90|\text{NO}) \\ &= 0,36 \cdot 0,6 \cdot 0,4 \cdot N_{\text{temp}}(87;74,6,7,89) \cdot N_{\text{hum}}(90;86,6,9,73) \\ &= 0.36 \cdot 0.6 \cdot 0.4 \cdot 0.014706 \cdot 0.038573 = 4.08\text{e-}05 \end{aligned}$$

- $\text{PLAY}=\text{sí}$

$$\begin{aligned} P(x|\text{Sí}) &= P(\text{NO}) \cdot P_{\text{O}}(\text{soleado}|\text{Sí}) \cdot P_{\text{w}}(\text{falso}|\text{Sí}) \cdot P_{\text{t}}(87|\text{Sí}) \cdot P_{\text{h}}(90|\text{Sí}) \\ &= 0.64 \cdot 0.22 \cdot 0.67 \cdot N_{\text{temp}}(87;73,6,16) \cdot N_{\text{hum}}(90;79.11,10.21) \\ &= 0.64 \cdot 0.22 \cdot 0.67 \cdot 0.004894 \cdot 0.022123 = 1.02\text{e-}05 \end{aligned}$$

- Normalizando:

$$P(\text{NO})=0,8 \quad P(\text{Sí}) = 0,2$$

Aprendizaje basado en instancias: Naïve Bayes

(Prácticas)

```
# Assigning features and label variables
weather=['Sunny','Sunny','Overcast','Rainy','Rainy','Rainy','Overcast','Sunny','S
'Rainy','Sunny','Overcast','Overcast','Rainy']
temp=['Hot','Hot','Hot','Mild','Cool','Cool','Cool','Mild','Cool','Mild','Mild','
play=['No','No','Yes','Yes','Yes','No','Yes','No','Yes','Yes','Yes','Yes','Yes',''
```

```
# Import LabelEncoder
from sklearn import preprocessing
#creating labelEncoder
le = preprocessing.LabelEncoder()
# Converting string labels into numbers.
weather_encoded=le.fit_transform(weather)
```

```
# Converting string labels into numbers
temp_encoded=le.fit_transform(temp)
label=le.fit_transform(play)
```

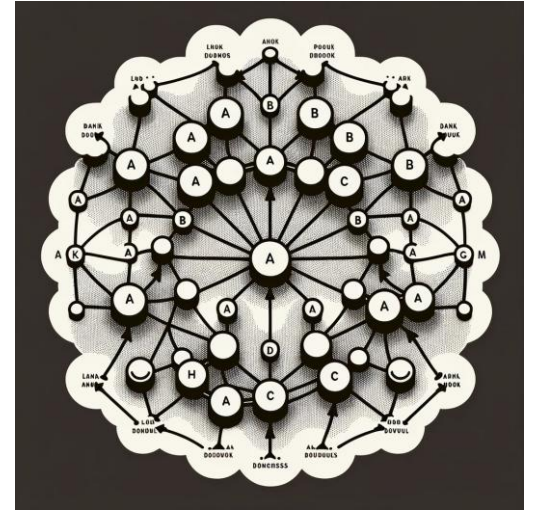
```
#Import Gaussian Naive Bayes model
from sklearn.naive_bayes import GaussianNB

#Create a Gaussian Classifier
model = GaussianNB()

# Train the model using the training sets
model.fit(features,label)

#Predict Output
predicted= model.predict([[0,2]]) # 0:Overcast, 2:Mild
print "Predicted Value:", predicted
```

```
#Combinig weather and temp into single listof tuples
features=zip(weather_encoded,temp_encoded)
```





Aprendizaje supervisado. Parte II

Introducción

Evaluación de modelos

Aprendizaje basado en instancias

Regresión Lineal

Modelo Naïve Bayes

Modelos basados en árboles de decisión

**Modelos multclasificadores y de
descomposición**